Portable Agent Skills as Interface Contracts A Technical Specification Method for Cross-Platform AI Skill Design

A Technical Specification Method for Cross-Platform AI Skill Design

Part 1 — From Prompt Templates to Skill Interface Engineering

---

Abstract

Modern AI development is moving from simple prompt usage toward reusable Agent Skills: structured capabilities that can analyze documents, operate tools, manage workflows, transform code, audit outputs, call external systems, and coordinate multi-stage reasoning. Yet complex Agent Skills are difficult to standardize because every platform has different assumptions about tools, memory, state, file access, orchestration, human approval, safety policy, and execution semantics.

The common mistake is to treat an Agent Skill as if it were only a better prompt template. This works for simple tasks such as summarization, rewriting, classification, or extraction. It fails for complex tasks that require multiple stages, intermediate artifacts, validation gates, audit traces, residual handling, and adaptation across different AI runtimes.

This article proposes a different approach:

```
A portable Agent Skill is not a universal prompt template; it is a declared interface contract whose invariant logic can survive platform-specific implementation. (0.1)
```

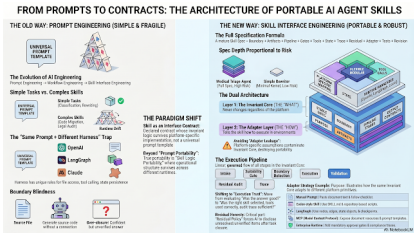
A **Technical Skill Specification** is the human-readable and machine-adaptable document that defines this interface contract. It declares the skill's purpose, non-purpose, input artifacts, output artifacts, runtime assumptions, pipeline stages, gates, tools, state policy, trace policy, residual audit, platform adapter mapping, test harness, failure modes, and revision rules.

The deeper conceptual foundation comes from Philosophical Interface Engineering, where an interface is understood as boundary, observables, gate, trace, residual, invariance, and revision. In that framework, a system becomes usable when its world of operation is declared clearly enough to be inspected, tested, corrected, and transferred.

This article applies that principle to AI engineering. It argues that the next stage of Agent Skill design is not merely prompt engineering, nor even workflow engineering, but **Skill Interface Engineering**.

```
Prompt Engineering → Workflow Engineering → Skill Interface Engineering. (0.2)
```

The result is a practical specification method for building Agent Skills that are portable, auditable, testable, residual-honest, and adaptable across platforms such as Codex-style Skills, Claude-style Skills, OpenAI Agents SDK workflows, LangGraph graphs, CrewAI processes, AutoGen conversations, MCP tool layers, A2A agent networks, and custom local LLM harnesses.



---

0. Reader's Guide: What This Article Is and Is Not

This article is written for AI engineers, agent framework designers, prompt engineers, technical writers, workflow architects, enterprise AI teams, and researchers interested in making complex AI capabilities reusable across platforms.

It is also written for people who sense that "prompt templates" are no longer enough.

A simple prompt can tell an AI what to do once. A complex Agent Skill must define how a class of tasks should be handled repeatedly, safely, inspectably, and adaptably.

This article is therefore about the missing middle layer between:

```
loose prompt
```

and:

```
full software implementation
```

That missing layer is the **Technical Skill Specification**.

0.1 What This Article Is

This article is a practical framework for writing technical specifications for complex Agent Skills.

It explains how to structure a Skill Spec so that another developer, another AI agent, or another platform adapter can implement the same underlying skill logic without losing its intended behavior.

It treats a Skill Spec as a portable declaration of:

```
task boundary;
input and output artifacts;
pipeline logic;
stage gates;
tool requirements;
state assumptions;
trace requirements;
residual audit;
platform adapter mapping;
test cases;
revision policy.
```

The core object is not a prompt.

The core object is an interface contract.

```
Technical Skill Specification = Portable Interface Contract for Reusable AI Capability. (0.3)
```

0.2 What This Article Is Not

This article is not a tutorial for one specific AI platform.

It does not claim that all Agent Skills should be implemented in one standard runtime.

It does not argue that every task needs a complex Skill Spec.

It does not replace ordinary prompting.

It does not claim that technical documentation alone is sufficient without implementation, testing, and governance.

It is also not a metaphysical theory of agents.

The goal is engineering discipline:

```
make complex Agent Skills easier to describe, port, test, govern, revise, and trust. (0.4)
```

0.3 The Central Shift

The central shift is from prompt portability to skill logic portability.

Many people ask:

```
Can we write one prompt that works everywhere?
```

But for complex Agent Skills, the better question is:

```
What invariant skill logic should survive when the implementation platform changes?
```

This produces the first structural distinction:

```
Prompt Portability = same instruction text works across contexts. (0.5)
```

```
Skill Logic Portability = same operational structure survives across runtimes. (0.6)
```

```
Interface Contract Portability = same declared boundary, pipeline, gate, trace, residual, and revision logic can be reimplemented through platform adapters. (0.7)
```

The article's core thesis can now be stated:

```
The most portable unit of complex Agent Skill design is not the prompt; it is the technical specification. (0.8)
```

Part I — The Problem: Why Complex Skills Cannot Be Universal Prompt Templates

1. The Failure of the Universal Skill Template

AI engineering began with prompts.

A prompt could say:

```
Summarize the following document.
```

or:

```
Rewrite this email professionally.
```

or:

```
Extract the key facts into a table.
```

For these tasks, a prompt template is often enough.

The task is small. The output is obvious. The runtime state is shallow. Tool usage is limited or absent. The user can easily judge the result. If the answer is not good, the user asks again.

But Agent Skills are becoming different.

A serious Agent Skill may need to:

```
read multiple files;
classify artifacts;
build an intermediate representation;
call tools;
write files;
run tests;
compare outputs;
detect missing information;
```

```
ask clarifying questions;
escalate to human approval;
produce an audit trace;
record unresolved residual;
adapt to different platform constraints.
```

At that point, the prompt-template model begins to fail.

1.1 Why Simple Prompt Templates Work for Simple Tasks

Simple prompt templates work when the task has four properties.

First, the task boundary is obvious.

Second, the input and output types are simple.

Third, the task does not require hidden intermediate artifacts.

Fourth, success can be judged directly from the final answer.

For example:

```
Summarize this article in five bullet points.
```

The boundary is clear: summarize this article.

The input is clear: article text.

The output is clear: five bullet points.

The success condition is clear enough: the summary should be accurate, concise, and relevant.

A reusable prompt template can work well here.

```
SimpleTask = ClearBoundary + SimpleInput + SimpleOutput + DirectEvaluation. (1.1)
```

Many prompt libraries are built around this kind of task.

They are useful.

But they should not be confused with Agent Skill architecture.

1.2 Why Complex Agent Skills Break the Template Model

A complex Agent Skill is different because the task is no longer a single transformation from input to output.

It is a governed sequence of transformations.

For example, consider a “legacy code migration” skill.

It may need to:

```
inspect source files;
identify the old runtime assumptions;
extract business logic;
map old functions to new functions;
generate target-language code;
run tests;
compare output behavior;
record mismatches;
revise implementation;
produce a final migration report.
```

A single prompt cannot reliably carry all this logic across different platforms, because each platform may differ in:

```
file access;
tool calling;
code execution;
state persistence;
working directory behavior;
approval policy;
logging;
memory;
error handling;
artifact creation;
security constraints.
```

The same prompt may behave differently because the harness is different.

```
Same Prompt + Different Harness → Different Skill Behavior. (1.2)
```

This is the root of the portability problem.

The prompt is not the whole skill.

The skill includes the prompt, but also the runtime world in which the prompt operates.

1.3 Same Intention, Different Harness Semantics

Suppose we define an Agent Skill:

```
Analyze a repository and produce a safe refactoring plan.
```

This sounds simple.

But each platform may interpret the task differently.

One platform may allow the agent to read all files.

Another may require explicit user approval before reading files.

One platform may allow code execution.

Another may allow only static analysis.

One platform may persist memory across sessions.

Another may discard all state after the response.

One platform may have built-in version control awareness.

Another may only see uploaded snippets.

One platform may allow editing.

Another may only generate suggestions.

Therefore, the same skill intention does not produce the same operational world.

```
SkillIntention ≠ SkillExecution. (1.3)
```

The missing layer is the declaration of runtime assumptions.

A Skill Spec must say:

```
This skill assumes file read access.
This skill does not assume persistent memory.
This skill requires user approval before destructive edits.
This skill produces a residual report when code execution is unavailable.
This skill must distinguish static recommendation from verified migration.
```

Without such declarations, “the same skill” is only a name.

1.4 The Universal Template Mistake

The universal template mistake occurs when we try to compress a complex Agent Skill into a single all-purpose prompt.

For example:

```
You are an expert AI agent. Analyze the task carefully, use available tools, follow best practices, validate your work, and produce a high-quality answer.
```

This kind of prompt may help a little.

But it is too vague for serious reuse.

It does not declare:

```
what tools are required;
what tools are optional;
what happens if tools are unavailable;
what intermediate artifacts must be produced;
what validation gates must pass;
what output contract is required;
what residual must be disclosed;
what human approval is needed;
what platform assumptions are allowed;
what failure modes are known.
```

It sounds mature, but it does not define the skill.

It produces a behavior style, not an interface contract.

```
UniversalSkillPrompt = Role + Intention + BestPracticeLanguage − RuntimeContract. (1.4)
```

The missing term is the runtime contract.

Without it, portability is superficial.

1.5 The Difference Between Skill Name and Skill World

A skill name is not a skill.

“Document analysis skill” is only a label.

“Code migration skill” is only a label.

“Legal reasoning skill” is only a label.

“Financial audit skill” is only a label.

A real skill requires a declared operational world.

It must say:

```
what counts as a document;
what counts as analysis;
what sources are admissible;
what evidence must be cited;
what uncertainty must be carried;
what outputs must be produced;
what validation must occur;
what residual remains after closure.
```

In other words:

```
SkillName + InterfaceContract → OperationalSkill. (1.5)
```

Without the interface contract, the skill name is an attractor word, not an engineering object.

It may make the AI sound more focused, but it does not make the skill portable.

1.6 Why “Just Use an Agent Framework” Is Not Enough

One may object:

```
Why not just use an agent framework?
```

Agent frameworks are useful. They provide orchestration, tool calling, memory, roles, workflows, events, state, and sometimes evaluation.

But a framework is not a specification.

A framework is one possible implementation environment.

A Skill Spec is the portable contract that can be implemented inside many environments.

The distinction is:

```
AgentFramework = Runtime for executing agent behavior. (1.6)
```

```
SkillSpec = Platform-neutral contract describing what behavior must be executed. (1.7)
```

A framework can run a skill.

It does not automatically define the skill's boundary, residual policy, trace requirements, test cases, or cross-platform invariant core.

Therefore, a mature team needs both:

```
SkillSpec + RuntimeFramework → ExecutableSkill. (1.8)
```

If the runtime changes, the Skill Spec should remain the reference.

1.7 The Real Engineering Problem

The real engineering problem is not:

```
How do we write a prompt that sounds intelligent?
```

The real problem is:

```
How do we define a reusable skill-world that remains stable when implemented across different AI runtimes?
```

This requires a different design discipline.

The Skill Spec must declare:

```
Boundary: what is inside and outside the task.
Observables: what the skill can inspect.
Gate: what counts as valid progress or completion.
Trace: what must be recorded for future inspection.
Residual: what remains unresolved.
Invariance: what must remain stable across platforms.
Revision: how the skill may evolve without losing accountability.
```

This is precisely where Agent Skill engineering meets interface engineering.

```
Agent Skill Engineering = Interface Declaration + Runtime Adaptation + Trace-Governed Execution. (1.9)
```

---

2. From Prompt Engineering to Skill Interface Engineering

Prompt engineering is not obsolete.

It remains useful.

But prompt engineering is only the first layer.

A prompt tells the model how to behave in a specific context.

A Skill Spec tells implementers how a reusable AI capability should exist across contexts.

This is a deeper object.

2.1 Prompt as Temporary Declaration

Every prompt declares a temporary world.

Even a simple prompt contains hidden assumptions.

For example:

```
Summarize this document for executives.
```

This declares:

```
audience = executives;
task = summary;
source = this document;
output style = concise and decision-oriented;
irrelevant details = probably omitted;
success = useful executive understanding.
```

The prompt is already an interface.

It declares what matters.

It creates a boundary.

It defines what the AI should see.

It sets a gate for acceptable output.

But ordinary prompts do this informally.

A Skill Spec does it explicitly.

```
Prompt = Informal Task Declaration. (2.1)
```

```
SkillSpec = Formalized Reusable Task Declaration. (2.2)
```

The difference is not only length.

The difference is governance.

2.2 Skill as Repeatable Operational World

A Skill is not merely a one-time instruction.

A Skill is a repeatable operational world.

For example, a spreadsheet analysis skill may repeatedly:

```
inspect workbook structure;
identify sheets and tables;
detect formulas;
summarize pivots;
validate calculations;
produce output files;
report assumptions;
list unresolved data issues.
```

A legal-document skill may repeatedly:

```
identify jurisdiction;
extract parties;
detect obligations;
map clauses;
classify risks;
separate facts from interpretation;
warn about missing legal advice;
produce residual issues.
```

A coding-agent skill may repeatedly:

```
read relevant files;
understand constraints;
plan changes;
modify code;
run tests;
inspect errors;
revise;
document known residual.
```

Each of these is a world with objects, rules, gates, traces, and residuals.

```
Skill = Repeatable Operational World for a Task Class. (2.3)
```

That world must be declared before it can be safely transferred.

2.3 Technical Documentation as the Portability Layer

When platforms differ, the most stable object is the technical document.

Not because documentation is more powerful than code.

But because documentation can describe the invariant logic above platform-specific execution.

The Skill Spec can say:

```
This stage requires file inspection.
On platform A, use the file-search tool.
On platform B, use repository context.
On platform C, ask the user to paste files.
On platform D, call a local script.
If no file inspection is available, return an incomplete-output residual.
```

The invariant requirement is:

```
inspect relevant source artifacts before making claims.
```

The adapter varies.

```
Invariant Requirement + Adapter Mapping → Platform-Specific Implementation. (2.4)
```

This is why a technical Skill Spec is often more portable than a prompt.

A prompt may collapse platform differences into vague language.

A Skill Spec exposes them.

2.4 The PIE Foundation: Boundary, Gate, Trace, Residual

Philosophical Interface Engineering provides a useful vocabulary for this engineering problem.

It says that a serious interface asks:

```
What is the boundary?
What is observable?
What passes the gate?
What trace is written?
What residual remains?
What survives reframing?
How can revision occur honestly?
```

The source framework compresses the method as:

```
Interface = Boundary + Observables + Gate + Trace + Residual + Invariance + Revision. (2.5)
```

For Agent Skills, the same formula becomes technical:

```
SkillInterface = TaskBoundary + RuntimeObservables + StageGates + ExecutionTrace + ResidualAudit + CrossPlatformInvariance + VersionRevision. (2.6)
```

This is the bridge from philosophy to AI engineering.

The problem is no longer abstract.

A Skill Spec is the technical form of this interface.

2.5 The Agent Skill as a Declared World

An Agent Skill must declare a world in which the AI can act responsibly.

That world includes:

```
objects;
inputs;
outputs;
tools;
permissions;
stages;
gates;
evidence;
memory;
failure modes;
human approval;
residual;
revision.
```

For example, a "research summary skill" must declare:

```
whether web search is allowed;
whether uploaded files are authoritative;
whether citations are required;
whether the answer should distinguish source fact from inference;
whether uncertainty should be surfaced;
whether recent information must be verified;
whether the output should include a residual section.
```

Without these declarations, the same skill may behave differently across runs.

```
Undeclared Skill → Runtime Drift. (2.7)
```

The Skill Spec prevents drift by making the task-world explicit.

```
Declared Skill → Governed Execution. (2.8)
```

2.6 From Persona to Procedure

Many prompts begin with a persona:

```
You are a senior analyst.
```

or:

```
You are an expert software architect.
```

Personas can help style and framing.

But they are not enough.

A persona does not define:

```
which inputs to inspect;
which tools to use;
which assumptions to reject;
which gates to apply;
which residuals to disclose;
which tests to run;
which artifacts to produce.
```

A Skill Spec moves from persona to procedure.

```
Persona Prompt = Identity Simulation. (2.9)
```

```
Skill Pipeline = Governed Transformation. (2.10)
```

The second is more portable.

A platform can implement a governed transformation even if it does not use the same persona language.

The procedure survives.

The persona may not.

2.7 From Workflow to Interface Contract

A workflow is closer to a Skill than a persona.

It defines stages.

For example:

```
Extract → Analyze → Draft → Validate → Output.
```

But a workflow is still not the whole Skill Spec.

A workflow says what steps occur.

A Skill Spec also says:

```
when the workflow should run;
what artifacts each step consumes;
what artifacts each step produces;
what gate each step must pass;
what happens if a step fails;
what tool is required;
what trace is written;
what residual remains;
how the workflow maps to different platforms;
how tests verify behavior.
```

Therefore:

```
Workflow = Ordered Steps. (2.11)
```

```
SkillSpec = Workflow + Boundary + Artifacts + Gates + Trace + Residual + Adapter + Tests + Revision. (2.12)
```

The workflow is inside the Skill Spec.

It is not the entire specification.

2.8 From Output Quality to Execution Trust

Ordinary prompt evaluation often asks:

```
Was the final answer good?
```

Skill evaluation must ask more:

```
Was the right skill selected?
Was the task boundary declared?
Were the required artifacts inspected?
Were the right tools used?
Were unsafe actions avoided?
Were validation gates passed?
Was residual disclosed?
Was the output contract satisfied?
Was enough trace left for debugging?
Would the same skill logic survive another platform?
```

This moves evaluation from output quality to execution trust.

```
OutputQuality = Quality(FinalAnswer). (2.13)
```

```
ExecutionTrust = Quality(FinalAnswer + Pipeline + Gates + Trace + Residual). (2.14)
```

A final answer may look good while the execution is untrustworthy.

For example, an AI may produce a polished migration plan without reading the relevant files.

The output is fluent.

The skill execution failed.

A Skill Spec makes such failure detectable.

---

3. What Is a Technical Skill Specification?

We can now define the central object.

A **Technical Skill Specification** is a platform-neutral document that declares how a reusable AI capability should operate.

It is not merely a prompt.

It is not merely code.

It is not merely a workflow diagram.

It is a structured contract that can be implemented in multiple runtimes.

3.1 Formal Definition

```
Technical Skill Specification = Platform-neutral description of a repeatable AI capability, including task boundary, artifacts, pipeline logic, gates, traces, residuals, adapter assumptions, tests, and revision rules. (3.1)
```

This definition has several important parts.

First, it is platform-neutral.

It should not assume one specific runtime unless it explicitly states an adapter profile.

Second, it describes a repeatable AI capability.

It is not written only for one task instance.

Third, it includes pipeline logic.

A Skill Spec is procedural, not merely descriptive.

Fourth, it includes gates.

The skill must know when to run, when to pause, when to ask, when to validate, and when to stop.

Fifth, it includes residual.

The skill must not hide unresolved uncertainty.

Sixth, it includes tests and revision.

A skill that cannot be tested or revised is not yet mature.

3.2 The Skill Spec as Interface Contract

A Skill Spec answers the following questions:

```
What does this Skill do?
What does this Skill refuse to do?
Who is the Skill for?
What inputs can it consume?
What outputs must it produce?
What intermediate artifacts should exist?
What tools does it require?
What state does it assume?
What memory does it write or read?
What gates must pass?
What trace must be left?
What residual must be disclosed?
How should it adapt to different platforms?
How should it be tested?
How should it be revised?
```

This makes the Skill Spec an interface contract.

It declares the operational world of the skill.

```
SkillSpec = Declared Operational World for a Reusable AI Capability. (3.2)
```

3.3 Skill Spec, Prompt, Workflow, Code

The distinction between prompt, workflow, code, and Skill Spec is essential.

A prompt is one instruction surface.

A workflow is one ordered execution pattern.

Code is one implementation.

A Skill Spec is the portable contract behind all of them.

```
Prompt = Runtime Instruction. (3.3)
```

```
Workflow = Ordered Execution Path. (3.4)
```

```
Code = Concrete Implementation. (3.5)
```

```
SkillSpec = Portable Contract Governing Possible Implementations. (3.6)
```

This means one Skill Spec can generate many implementation forms.

For example:

```
one-page manual checklist;
Claude-style SKILL.md;
Codex-style Skill folder;
OpenAI Agents SDK workflow;
LangGraph graph;
CrewAI process;
AutoGen multi-agent conversation;
MCP prompt and tool package;
local Ollama harness;
enterprise governed runtime.
```

Each implementation may differ.

The Skill Spec remains the reference.

3.4 The Compact Formula

A mature Skill Spec can be compressed into one formula:

```
SkillSpec = Boundary + Artifacts + Pipeline + Gates + Tools + State + Trace + Residual + Adapter + Tests + Revision. (3.7)
```

Each term is necessary.

If boundary is missing, the skill overreaches.

If artifacts are missing, the skill cannot know what it consumes or produces.

If pipeline is missing, the skill becomes a persona.

If gates are missing, the skill cannot control commitment.

If tools are missing, runtime assumptions stay hidden.

If state is missing, memory behavior becomes unstable.

If trace is missing, the skill cannot be audited.

If residual is missing, the skill becomes overconfident.

If adapter is missing, portability is false.

If tests are missing, quality cannot be reproduced.

If revision is missing, the skill cannot evolve responsibly.

3.5 The Minimal Skill Spec

Not every Skill Spec must be huge.

A minimal Skill Spec may contain only:

```
Skill name;
purpose;
non-purpose;
input types;
output contract;
pipeline steps;
gates;
residual policy;
platform assumptions;
test examples.
```

This may be enough for a small team.

The minimal formula is:

```
MinimalSkillSpec = Purpose + Boundary + Pipeline + Output + Residual + Tests. (3.8)
```

But for enterprise-grade or high-risk skills, the full specification is better.

The complexity of the Skill Spec should match the risk and complexity of the skill.

```
SpecDepth ∝ SkillRisk + WorkflowComplexity + PortabilityNeed. (3.9)
```

A simple rewrite skill does not need an enterprise specification.

A legal document review skill probably does.

A code migration skill often does.

A medical triage assistant certainly does.

A financial transaction agent definitely does.

3.6 Why This Is Not Bureaucracy

Some engineers may worry that this approach creates too much documentation.

That is a real risk.

A Skill Spec should not become paperwork for its own sake.

The purpose is not bureaucratic completeness.

The purpose is operational clarity.

A good Skill Spec reduces confusion, prevents hidden assumptions, improves portability, enables testing, and makes failure visible.

```
GoodSpecification = LessRuntimeAmbiguity + BetterPortability + EasierTesting + SaferRevision. (3.10)
```

A bad specification is long but useless.

A good specification may be short but decisive.

The test is simple:

```
Does this document help another agent or developer reproduce the skill safely? (3.11)
```

If yes, it is useful.

If no, it is decoration.

3.7 The Skill Spec as Source of Truth

In a mature AI engineering team, the Skill Spec should become the source of truth for the skill.

Prompts may change.

Tool APIs may change.

Frameworks may change.

Model behavior may change.

But the Skill Spec says what the skill is supposed to do.

When output drifts, the team compares it to the Skill Spec.

When porting to a new platform, the team maps the Skill Spec into the new runtime.

When a failure occurs, the team records whether the failure was due to:

```
bad specification;
bad adapter;
bad prompt;
bad model behavior;
bad tool execution;
bad state handling;
bad validation;
missing residual policy.
```

This creates a disciplined debugging structure.

```
SkillFailureDiagnosis = Compare(RuntimeBehavior, SkillSpec). (3.12)
```

Without a Skill Spec, every failure becomes vague.

With a Skill Spec, failure can be localized.

3.8 The First Principle

We can now state the first principle of portable Agent Skill design:

```
No complex Agent Skill should be treated as portable until its invariant logic has been separated from its platform adapter. (3.13)
```

This principle leads directly to Part 2:

```
Invariant Core and Adapter Layer.
```

That is where portability becomes explicit.

Part II — The Core Architecture of a Portable Skill Spec

4. The Invariant Core and the Adapter Layer

A portable Agent Skill has two layers.

The first layer is the **Invariant Core**.

The second layer is the **Adapter Layer**.

The Invariant Core defines what the Skill is, regardless of platform.

The Adapter Layer defines how the Skill becomes executable inside a specific runtime.

This distinction is the heart of cross-platform Agent Skill design.

```
PortableSkill = InvariantCore + VariableAdapterLayer. (4.1)
```

Without this distinction, a team will confuse platform behavior with skill logic.

That confusion is the beginning of false portability.

4.1 The Invariant Core

The Invariant Core contains the logic that should remain stable across platforms.

It answers:

```
What task does this Skill solve?
What inputs does it accept?
What outputs does it produce?
What pipeline must it follow?
What gates must it pass?
What residual must it disclose?
What validation must occur?
What safety or authority rules must always apply?
```

For example, a document analysis Skill may have the following invariant core:

```
1. Read the user's declared document set.
2. Identify the document type and scope.
3. Extract major claims.
4. Separate source facts from interpretation.
5. Cite or reference evidence when available.
6. Produce a structured summary.
7. List unresolved residual.
8. Avoid claims not supported by the supplied documents unless external search is explicitly enabled.
```

This logic should remain stable whether the Skill is implemented in:

```
a manual prompt;
a Codex-style Skill;
a Claude-style Skill;
an OpenAI Agents SDK workflow;
a LangGraph graph;
a CrewAI process;
an MCP-connected tool layer;
a local Ollama harness;
an enterprise internal agent platform.
```

The runtime may change.

The invariant logic should not.

```
InvariantCore = Purpose + Boundary + Pipeline + Gates + OutputContract + ResidualPolicy + TestLogic. (4.2)
```

4.2 What Belongs in the Invariant Core

The Invariant Core should include at least seven elements.

1. Skill Purpose

What repeatable capability does the Skill provide?

Example:

```
This Skill converts legacy report logic into a target-language implementation and produces an equivalence audit.
```

2. Skill Boundary

What is inside and outside scope?

Example:

```
Inside scope: syntax conversion, semantic mapping, generated implementation, comparison report.
Outside scope: production deployment, legal approval, unverified business-rule invention.
```

3. Pipeline Logic

What stages must occur?

Example:

```
Intake → Parse → Map → Generate → Compare → Validate → Report → Residual.
```

4. Gate Rules

What conditions must be satisfied before moving forward?

Example:

```
Do not generate final migration output until all required source artifacts have been inspected or explicitly marked unavailable.
```

5. Output Contract

What must the final output contain?

Example:

```
Generated code, mapping notes, equivalence status, unresolved mismatches, test instructions.
```

6. Residual Policy

What uncertainty must be disclosed?

Example:

```
If source behavior cannot be verified, mark the output as unverified and list the missing evidence.
```

7. Test Logic

How should the Skill be evaluated?

Example:

```
Run golden-path conversion, ambiguous-input test, missing-file test, and equivalence-failure test.
```

Together:

```
InvariantCore = Purpose + Boundary + Pipeline + GateRules + OutputContract + ResidualPolicy + TestLogic. (4.3)
```

4.3 The Adapter Layer

The Adapter Layer converts the Invariant Core into platform-specific execution.

It answers:

```
How does this runtime read files?
How does it call tools?
How does it store state?
How does it ask for approval?
How does it record trace?
How does it handle errors?
How does it expose residual?
How does it produce artifacts?
```

For example, the same "document analysis" Skill may be adapted differently.

Runtime	Adapter strategy
Manual prompt	Paste document text and follow checklist
Codex-style Skill	Use `SKILL.md`, repository files, scripts
Claude-style Skill	Use Skill folder, instructions, resources
Agents SDK	Use agent instructions, tools, guardrails, handoffs
LangGraph	Use nodes, edges, state object, checkpoints
MCP	Expose document resources and prompt templates
Local LLM harness	Use local file loader, prompt kernel, output parser
Enterprise runtime	Add approval gate, audit logs, compliance trace

Formula:

```
Adapter = Map(InvariantCore → PlatformPrimitives). (4.4)
```

The Adapter Layer should not redefine the Skill's purpose.

It should only express the same purpose through a platform's available primitives.

4.4 Adapter Leakage

Adapter leakage happens when platform-specific assumptions contaminate the Invariant Core.

Example:

```
The Skill assumes every platform has persistent memory.
```

This is dangerous because many platforms do not.

Another example:

```
The Skill assumes code execution is always available.
```

This is also dangerous because some platforms only allow static reasoning.

Another example:

```
The Skill assumes all files are visible to the model.
```

This may fail in systems where file access requires explicit search, retrieval, or user selection.

Adapter leakage creates false portability.

```
AdapterLeakage = PlatformSpecificAssumption treated as InvariantSkillLogic. (4.5)
```

The cure is to separate:

```
Must always happen
```

from:

```
How it happens on this platform
```

For example:

```
Invariant: Relevant source files must be inspected before final claims.
Adapter A: Use repository read access.
Adapter B: Use file search.
Adapter C: Ask user to upload files.
Adapter D: Mark output incomplete if files cannot be inspected.
```

This separation is the essence of portable specification.

4.5 False Portability

False portability occurs when a Skill appears reusable but actually depends on hidden platform behavior.

A prompt may say:

```
Use available tools to verify your work.
```

This looks portable.

But it hides several questions:

```
Which tools are available?
Can the agent choose tools autonomously?
Can tools read files?
Can tools write files?
Can tools execute code?
Can tools access the web?
Can tools call external APIs?
Are tool results persisted?
Are tool errors visible?
```

A better Skill Spec says:

```
Verification requires at least one of the following:
1. executable test run;
2. static comparison against expected output;
3. user-provided validation fixture;
4. explicit residual statement that verification was not possible.
```

This is portable because it defines the verification requirement abstractly and allows several adapter paths.

```
TruePortability = InvariantRequirement + MultipleValidAdapterPaths + ResidualIfUnavailable. (4.6)
```

4.6 Invariance as the Test of Portability

A Skill is portable only if its essential behavior survives reframing into another runtime.

The question is:

```
If this Skill is implemented on another platform, does it still preserve its task boundary, pipeline, gates, output contract, and residual policy?
```

If yes, it has real portability.

If no, it only has surface portability.

```
SkillPortability = Stability(InvariantCore) under PlatformReframing. (4.7)
```

This gives us a practical test:

```
Can another team implement this Skill from the specification without relying on undocumented assumptions?
```

If not, the Skill Spec is incomplete.

---

5. Boundary Declaration

Every Skill begins with a boundary.

Boundary is the first act of technical world-making.

A Skill without boundary will overreach, hallucinate, misuse tools, or produce outputs that sound plausible but exceed its authority.

```
SkillBoundary = Purpose + NonPurpose + InputScope + OutputScope + RiskLimit + HumanEscalationRule. (5.1)
```

Boundary declaration answers:

```
What is this Skill allowed to do?
What is it not allowed to do?
What evidence may it use?
What outputs may it produce?
What decisions must remain with the user?
When must it refuse, pause, or escalate?
```

5.1 Purpose Boundary

The purpose boundary defines the positive scope of the Skill.

Example:

```
This Skill helps convert legacy business-report logic into a target JavaScript implementation and produces an equivalence audit.
```

This purpose is specific.

It is not:

```
This Skill helps with programming.
```

That is too broad.

A strong purpose boundary should identify:

```
task class;
target artifact;
expected transformation;
success condition.
```

Formula:

```
PurposeBoundary = TaskClass + TargetArtifact + Transformation + SuccessCondition. (5.2)
```

Example:

```
TaskClass = legacy report migration.
TargetArtifact = JavaScript implementation.
Transformation = convert source logic into target code.
SuccessCondition = generated behavior is equivalent or residual mismatch is disclosed.
```

5.2 Non-Purpose Boundary

The non-purpose boundary is equally important.

It prevents the Skill from expanding into adjacent but unsafe or unsupported territory.

Example:

```
This Skill does not certify production readiness.
This Skill does not make legal, financial, or regulatory judgments.
This Skill does not deploy code.
This Skill does not invent missing business rules.
This Skill does not modify external systems without explicit approval.
```

A good Skill Spec must say what the Skill refuses.

```
NonPurposeBoundary = ExplicitRefusalZone + EscalationZone + UnsupportedAssumptionZone. (5.3)
```

Without non-purpose, the Skill becomes elastic.

Elastic Skills are dangerous because they can sound competent beyond their declared authority.

5.3 Input Scope

The input scope defines what the Skill can consume.

For example:

```
Accepted inputs:
- user request;
- source files;
- target language requirements;
- test cases;
- expected output examples;
- platform constraints;
- user-provided business rules.
```

It should also define invalid or incomplete inputs:

```
Incomplete inputs:
- source code missing;
- target runtime unspecified;
- no equivalence criteria;
- no sample data;
- unclear output format.
```

The Skill does not always need to stop when input is incomplete.

But it must know how to mark incompleteness.

```
IncompleteInput → Clarify | ProceedWithResidual | Refuse | Escalate. (5.4)
```

5.4 Output Scope

The output scope defines what the Skill may produce.

Examples:

```
final answer;
technical report;
generated code;
configuration file;
test plan;
audit table;
residual register;
implementation notes;
adapter mapping.
```

It should also define what it must not output.

Example:

```
The Skill must not output a verified-equivalence claim unless comparison evidence exists.
```

This matters because AI systems often over-close.

They produce final-sounding answers even when verification is incomplete.

A good output scope prevents false closure.

```
OutputScope = AllowedArtifacts + RequiredArtifacts + ForbiddenClaims + IncompleteOutputRules. (5.5)
```

5.5 Risk Boundary

The risk boundary defines when the Skill must slow down, refuse, or ask for human approval.

Risk boundaries are especially important for Skills involving:

```
legal analysis;
medical advice;
financial decisions;
external actions;
public release;
code deployment;
data deletion;
personal data;
security-sensitive systems;
memory writes;
irreversible tool calls.
```

A risk boundary may define categories:

```
Low risk: summarize, classify, reformat.
Medium risk: generate draft, propose code, suggest workflow.
High risk: execute action, modify system, advise regulated decision.
Critical risk: irreversible external action or safety-sensitive judgment.
```

Formula:

```
RiskBoundary = RiskClass + AllowedAction + RequiredGate + RequiredResidual. (5.6)
```

Example:

```
If action affects external systems, require explicit human approval before execution.
```

5.6 Human Escalation Rule

Some tasks cannot be fully delegated to a Skill.

The Skill Spec should define when human judgment is required.

Examples:

```
missing source authority;
ambiguous legal interpretation;
unsafe external action;
conflicting user instructions;
irreversible edit;
production deployment;
medical or financial decision;
unresolvable platform limitation.
```

Human escalation is not a weakness.

It is part of the boundary.

```
HumanEscalation = BoundaryCondition where AI assistance must stop short of final authority. (5.7)
```

A mature Skill knows its authority limit.

5.7 Boundary Failure Modes

Boundary failure is one of the most common Skill failures.

1. Boundary Overreach

The Skill answers outside its scope.

Example:

```
A document-summary Skill gives legal advice based on a contract.
```

2. Boundary Collapse

The Skill treats all tasks as equivalent.

Example:

```
A code-review Skill starts rewriting architecture, security policy, and deployment workflows without being asked.
```

3. Boundary Blindness

The Skill does not notice missing inputs.

Example:

```
A migration Skill generates code without the source file.
```

4. Boundary Drift

The Skill gradually changes purpose across turns.

Example:

```
A documentation Skill becomes a decision-making Skill.
```

5. Boundary Capture

The Skill allows user pressure to override its declared safety limits.

Example:

```
The user asks the Skill to skip validation and present unverified output as final.
```

Formula:

```
BoundaryFailure = Overreach + Collapse + Blindness + Drift + Capture. (5.8)
```

A good Skill Spec should list known boundary failures and how to handle them.

---

6. Artifact Ontology

A complex Agent Skill does not operate only on text.

It operates on artifacts.

An artifact is any object the Skill consumes, produces, transforms, stores, validates, or cites.

```
Artifact = Any input, intermediate object, output, trace, or residual used by the Skill. (6.1)
```

A Skill Spec must define its artifact ontology because different platforms represent artifacts differently.

One platform may treat a file as context.

Another may treat it as a searchable resource.

Another may require a tool call.

Another may require manual upload.

Another may serialize it into state.

Without artifact ontology, the Skill cannot be ported reliably.

6.1 Input Artifacts

Input artifacts are objects the Skill receives or retrieves before processing.

Common input artifacts include:

```
user request;
uploaded file;
document text;
spreadsheet;
slide deck;
PDF;
code repository;
database schema;
API specification;
tool output;
conversation history;
configuration file;
test fixture;
external policy;
human instruction;
approval signal.
```

The Skill Spec should define for each input type:

```
name;
format;
required or optional;
source;
validation rule;
failure behavior.
```

Example:

```
InputArtifact: SourceReportFile
Format: .sql, .js, .json, .txt, or exported legacy report file
Required: yes
Validation: must be readable and relevant to declared report ID
Failure behavior: ask user to provide file or proceed with residual if partial analysis is acceptable
```

Formula:

```
InputArtifactSpec = Name + Format + RequiredFlag + Source + ValidationRule + FailureBehavior. (6.2)
```

6.2 Intermediate Artifacts

Intermediate artifacts are working objects created during the Skill pipeline.

They are often invisible in simple prompting, but essential in complex Skills.

Examples:

```
intent map;
source inventory;
claim map;
field mapping table;
execution plan;
risk register;
tool plan;
draft output;
validation notes;
comparison table;
error list;
residual register;
adapter map.
```

Intermediate artifacts matter because they make the Skill inspectable.

A final answer may be wrong.

The intermediate artifacts help locate where the error occurred.

```
IntermediateArtifacts = DebuggableState of SkillExecution. (6.3)
```

A Skill Spec should define which intermediate artifacts are mandatory.

Example:

```
A code migration Skill must produce a mapping table before final generated code.
```

This prevents the Skill from jumping directly to output.

6.3 Output Artifacts

Output artifacts are the final deliverables.

Examples:

```
answer;
report;
generated code;
configuration file;
spreadsheet;
diagram;
test suite;
migration plan;
audit log;
JSON result;
YAML specification;
residual footer;
human review checklist.
```

A Skill Spec should define:

```
required outputs;
optional outputs;
format;
audience;
validation rule;
residual requirement.
```

Example:

```
OutputArtifact: MigrationAuditReport
Format: Markdown table and narrative summary
Required: yes
Validation: must list converted items, equivalence status, mismatch items, and unresolved residual
```

Formula:

```
OutputArtifactSpec = Name + Format + Audience + RequiredFlag + ValidationRule + ResidualRule. (6.4)
```

6.4 Audit Artifacts

Audit artifacts are records that help another person or system inspect the Skill run.

Examples:

```
tool-call summary;
file-read summary;
assumption list;
validation results;
test outcomes;
human approvals;
decision log;
error log;
source-to-output mapping;
version information.
```

Audit artifacts should answer:

```
What did the Skill inspect?
What did it assume?
What did it generate?
What did it validate?
What failed?
What remains unresolved?
```

Formula:

```
AuditArtifact = Evidence of SkillExecutionPath. (6.5)
```

In low-risk tasks, audit artifacts may be lightweight.

In high-risk tasks, they are essential.

6.5 Residual Artifacts

Residual artifacts record what remains unresolved.

They may include:

```
missing file list;
uncertain claim list;
unsupported inference list;
tool failure note;
conflicting evidence table;
user clarification needed;
platform limitation;
human review required;
unverified output warning.
```

A residual artifact prevents false finality.

It says:

```
This Skill has closed what it can close, but these items remain open.
```

Formula:

```
ResidualArtifact = Structured Unfinished Material after Skill Closure. (6.6)
```

Every complex Skill should produce residual artifacts when uncertainty remains.

6.6 Artifact Lineage

Artifact lineage records how outputs were produced from inputs.

For example:

```
Input SQL file → Parsed condition map → Template mapping → Generated SQL → Comparison report → Residual mismatch list.
```

This is especially important in migration, compliance, legal, finance, research, and high-stakes documentation tasks.

```
ArtifactLineage = Ordered Relation from InputArtifacts to OutputArtifacts through IntermediateArtifacts. (6.7)
```

Without artifact lineage, debugging becomes difficult.

With lineage, a reviewer can ask:

```
Which source produced this output?
Which stage transformed it?
Which gate approved it?
Which residual remains attached?
```

This is the difference between fluent output and governed output.

---

7. Runtime Assumptions

A Skill Spec must declare runtime assumptions.

Runtime assumptions define what the Skill expects the platform to provide.

If assumptions are hidden, portability fails.

```
RuntimeAssumptions = ToolAccess + FileAccess + StatePolicy + MemoryPolicy + ExecutionPolicy + ApprovalPolicy + LoggingPolicy. (7.1)
```

7.1 Tool Assumptions

The Skill Spec should state what tools are required, optional, or forbidden.

Example:

```
Required:
- file reading;
- text search;
- output writing.

Optional:
- code execution;
- web search;
- diagram generation.

Forbidden:
- external system modification without approval;
- destructive file operations;
- unsupported financial or legal execution.
```

The Skill should also say what happens when a tool is unavailable.

```
ToolUnavailable → Fallback | PartialOutput | AskUser | Refuse | Escalate. (7.2)
```

A serious Skill must not pretend to verify something if the verification tool is unavailable.

7.2 File Access Assumptions

File handling differs sharply across platforms.

A Skill Spec should define:

```
Can the Skill read uploaded files?
Can it search files?
Can it inspect repository directories?
Can it write new files?
Can it overwrite files?
Can it create downloadable artifacts?
Does it need user approval before editing?
```

Example declaration:

```
This Skill assumes read-only access to declared input files. It does not assume permission to overwrite source files. Generated files must be written as new artifacts unless the user explicitly requests replacement.
```

Formula:

```
FilePolicy = ReadScope + SearchScope + WriteScope + OverwriteRule + ApprovalRule. (7.3)
```

7.3 Memory Assumptions

Memory is one of the most dangerous hidden assumptions in Agent Skill design.

Some platforms have no persistent memory.

Some have session memory.

Some have user profile memory.

Some have project memory.

Some have vector-store retrieval.

Some have explicit state objects.

Some have hidden conversation context.

A Skill Spec must say what kind of memory it assumes.

Possible memory modes:

```
NoMemory;
SessionMemory;
TaskState;
ProjectMemory;
PersistentUserMemory;
```

```
ExternalKnowledgeStore;
AuditLedgerMemory.
```

Formula:

```
MemoryMode ∈ {None, Session, TaskState, Project, Persistent, ExternalStore, AuditLedger}. (7.4)
```

A Skill that requires persistent memory cannot be ported to a stateless runtime without an adapter.

The adapter must provide external state or mark the Skill as reduced capability.

7.4 State Assumptions

State is not the same as memory.

Memory is what is remembered across time.

State is the active working condition of the current run.

Examples:

```
current stage;
selected files;
parsed entities;
tool results;
validation status;
pending approvals;
open residual;
retry count;
current output draft.
```

A Skill Spec should define whether state is:

```
implicit in conversation;
stored in a structured object;
stored in files;
stored in graph nodes;
stored in database;
not stored.
```

Formula:

```
SkillState = CurrentStage + WorkingArtifacts + GateStatus + ToolResults + ResidualStatus. (7.5)
```

Complex Skills should prefer explicit state.

Implicit state is convenient but fragile.

7.5 Execution Assumptions

Execution assumptions define what the AI can do.

Examples:

```
Can it execute code?
Can it run shell commands?
Can it call APIs?
Can it modify files?
Can it send emails?
Can it schedule tasks?
Can it call other agents?
Can it ask the user for approval?
Can it stop and resume later?
```

A Skill Spec must distinguish between:

```
recommendation;
draft creation;
simulation;
verified execution;
external action.
```

These are different levels of authority.

```
ExecutionLevel = Recommend | Draft | Simulate | Verify | ActExternally. (7.6)
```

A Skill may be safe at the recommendation level but unsafe at the external-action level.

7.6 Human Approval Assumptions

A mature Skill must define when human approval is required.

Human approval may be needed before:

```
sending email;
deleting files;
overwriting files;
publishing content;
submitting forms;
making purchases;
creating calendar events;
changing settings;
modifying production systems;
storing sensitive memory;
making regulated decisions.
```

Formula:

```
ApprovalGate(Action) = Required if Action is Irreversible, External, Sensitive, Regulated, or User-Identity-Binding. (7.7)
```

This should be part of the Skill Spec, not left to vague judgment.

7.7 Logging and Trace Assumptions

The Skill Spec should state what level of trace is required.

Possible trace levels:

```
None;
UserVisibleSummary;
InternalDebugLog;
StructuredAuditTrail;
ComplianceGradeTrace;
ReproducibleExecutionLedger.
```

Formula:

```
TraceLevel ∈ {None, Summary, DebugLog, AuditTrail, ComplianceTrace, ReproducibleLedger}. (7.8)
```

A casual summarization Skill may only need a user-visible summary.

A code migration Skill may need an audit trail.

A regulated workflow may need compliance-grade trace.

The required trace level should match the risk.

```
RequiredTraceLevel ∝ RiskLevel + ReproducibilityNeed + HumanReviewNeed. (7.9)
```

7.8 Platform Independence Statement

Every Skill Spec should include a platform independence statement.

Example:

```
This Skill is defined at the interface-contract level. The invariant logic consists of its purpose, boundary, pipeline, gates, output contract, residual policy, and test cases. Tool use, memory implementation, file access, and logging are adapter-specific and must be mapped into the target runtime.
```

This statement prevents confusion.

It tells implementers which parts are sacred and which parts are replaceable.

```
PlatformIndependence = InvariantCore declared separately from RuntimeAdapter. (7.10)
```

7.9 Runtime Assumption Failure

Runtime assumption failure occurs when the Skill expects something the platform cannot provide.

Common cases:

```
Skill expects file access, but files are unavailable.
Skill expects code execution, but runtime is text-only.
Skill expects persistent memory, but platform is stateless.
Skill expects human approval hooks, but platform has no approval flow.
Skill expects tool calls, but tool use is disabled.
Skill expects audit logging, but no trace store exists.
```

A good Skill Spec should define fallback behavior.

```
RuntimeFailure = MissingAssumption + NoFallback + HiddenResidual. (7.11)
```

A mature Skill never hides runtime failure.

It says:

```
This output is incomplete because the required runtime capability was unavailable.
```

That is not a defect.

It is residual honesty.

Closing of Part II

Part II has defined the core architecture of a portable Skill Spec.

The essential distinction is:

```
Invariant Core = what the Skill must preserve.
Adapter Layer = how a platform implements it.
```

A serious Skill Spec must declare:

```
boundary;
artifacts;
runtime assumptions;
tool policy;
file policy;
memory policy;
state policy;
approval policy;
trace policy.
```

This gives us a stable foundation.

The next part turns the specification into a living execution structure:

```
Pipeline Logic and Gate Stack.
```

That is where the Skill stops being a document and becomes an executable operational world.

Part III — Pipeline Logic and Gate Stack

8. Pipeline Logic

A complex Agent Skill must be described as a pipeline.

A pipeline is not merely a list of steps. It is a governed transformation path.

It defines how the Skill moves from raw user request to inspected input, from inspected input to intermediate representation, from intermediate representation to output, from output to validation, and from validation to residual disclosure.

```
SkillPipeline = Intake → Gate → Parse → Plan → Execute → Validate → Output → Audit → Trace → Revise. (8.1)
```

This formula is deliberately generic. Different Skills will instantiate it differently.

A research Skill may use:

```
Question → Source Search → Source Filtering → Evidence Map → Synthesis → Citation Audit → Residual.
```

A code migration Skill may use:

```
Source Intake → Parse → Semantic Map → Generate → Run Tests → Compare → Fix → Residual.
```

A document-review Skill may use:

```
File Intake → Classification → Clause Extraction → Risk Mapping → Evidence Table → Review Notes → Residual.
```

A spreadsheet Skill may use:

```
Workbook Intake → Sheet Inventory → Formula/Pivot Inspection → Data Model → Analysis → Output File → Validation.
```

The exact steps differ. The deeper structure is the same.

A Skill is a governed movement from uncertainty toward usable closure.

```
SkillExecution = Controlled Closure over a Declared Task World. (8.2)
```

8.1 Why a Skill Is a Pipeline, Not a Persona

Many AI prompts still begin with role language:

```
You are an expert analyst.
```

or:

```
You are a senior software engineer.
```

This can be useful for style and expectation, but it is not enough for repeatable Skill behavior.

A persona tells the model what kind of voice or expertise to simulate.

A pipeline tells the model what transformations must occur.

```
Persona = Simulated Role. (8.3)
```

```
Pipeline = Ordered Transformation with Gates and Trace. (8.4)
```

For simple tasks, persona may be enough.

For complex Skills, persona is too weak.

A "senior engineer" persona does not guarantee that the agent will inspect the correct files, run the correct tests, preserve the correct residual, or produce the correct audit artifacts.

A pipeline can require those things.

Therefore, the Skill Spec should not merely say:

```
Act like an expert.
```

It should say:

```
Perform these stages, produce these artifacts, pass these gates, disclose these residuals.
```

That is the difference between role prompting and Skill engineering.

8.2 The Generic Pipeline Model

A general Agent Skill pipeline can be written as:

```
Intake → Suitability Gate → Intent Extraction → Boundary Detection → Artifact Inventory → Tool Planning → Execution → Validation → Output Assembly → Residual Audit → Trace Writing → Revision Path. (8.5)
```

Each stage has a different function.

Stage	Function
Intake	Receive user request and available artifacts
Suitability Gate	Decide whether this Skill should run
Intent Extraction	Identify what the user is really asking
Boundary Detection	Define scope, non-scope, risk, and authority
Artifact Inventory	Identify required and available inputs
Tool Planning	Decide what tools are needed
Execution	Perform the main transformation
Validation	Check correctness or contract satisfaction
Output Assembly	Produce user-facing or machine-readable output
Residual Audit	List unresolved uncertainty, missing data, or limitations
Trace Writing	Record useful execution history
Revision Path	Define next correction or improvement if needed

This pipeline is not mandatory for every Skill. But it is a strong default.

A simple Skill may compress stages.

A high-risk Skill should expand them.

```
PipelineDepth ∝ SkillComplexity + RiskLevel + AuditNeed. (8.6)
```

8.3 Stage Specification

Each pipeline stage should be specified in the Skill Spec.

A good stage definition should include:

```
stage name;
purpose;
input artifacts;
operation;
output artifacts;
gate condition;
failure behavior;
trace item.
```

Formula:

```
StageSpec = Name + Purpose + Input + Operation + Output + Gate + FailureBehavior + TraceItem. (8.7)
```

Example:

```
Stage Name: Artifact Inventory
Purpose: Identify all required and available source artifacts.
Input: user request, uploaded files, repository context.
Operation: classify files by role and relevance.
Output: artifact inventory table.
Gate: required artifacts are available or marked missing.
Failure Behavior: ask user for missing files or proceed with residual.
Trace Item: list of files inspected and missing artifacts.
```

This kind of stage definition makes the Skill portable.

Another runtime may implement the operation differently, but the stage contract remains stable.

8.4 Linear, Branching, Iterative, and Recursive Pipelines

Not all Skills are linear.

A simple summary Skill may be linear:

```
Input → Summarize → Output.
```

A document review Skill may branch:

```
If contract → clause review.
If policy document → compliance review.
If research paper → argument map.
If unknown type → classification residual.
```

A code migration Skill may iterate:

```
Generate → Test → Compare → Fix → Test Again.
```

A research Skill may recursively refine:

```
Search → Read → Identify gap → Search again → Synthesize → Residual.
```

Therefore, a Skill Spec should define the pipeline type.

```
PipelineType ∈ {Linear, Branching, Iterative, Recursive, Hybrid}. (8.8)
```

Each type has different risks.

Linear pipelines may over-close too early.

Branching pipelines may choose the wrong path.

Iterative pipelines may loop without progress.

Recursive pipelines may drift from the original question.

Hybrid pipelines need stronger trace.

8.5 Pipeline State

A complex pipeline needs state.

State records where the Skill currently is.

Possible state fields:

```
current_stage;
selected_pipeline_path;
available_artifacts;
missing_artifacts;
tool_results;
validation_status;
open_residual;
human_approval_status;
retry_count;
output_draft;
error_list.
```

Formula:

```
PipelineState = CurrentStage + Path + Artifacts + ToolResults + Gates + Residual + OutputDraft. (8.9)
```

A platform may store this state in different ways:

```
conversation context;
graph state;
JSON object;
temporary file;
database record;
memory store;
agent scratchpad.
```

The Skill Spec should not assume one storage method unless it is platform-specific.

It should define the logical state first.

Then the adapter maps logical state to runtime state.

```
LogicalState → AdapterState. (8.10)
```

8.6 Pipeline Compression

Some platforms cannot support long pipeline instructions.

Some contexts are token-limited.

Some Skills need both a full specification and a compact runtime kernel.

Therefore, a Skill Spec may define three levels:

```
Full Pipeline;
Runtime Pipeline;
Minimal Kernel.
```

The Full Pipeline is for documentation and implementation.

The Runtime Pipeline is for actual agent instructions.

The Minimal Kernel is for compressed execution.

Example:

```
Full: Intake → Suitability Gate → Intent Extraction → Boundary Detection → Artifact Inventory → Tool Planning → Execution → Validation → Output Assembly → Residual Audit → Trace Writing → Revision Path.

Runtime: Intake → Plan → Execute → Validate → Residual.

Kernel: Declare boundary, inspect artifacts, execute pipeline, validate output, disclose residual.
```

Formula:

```
PipelineCompression = FullSpec → RuntimeSpec → KernelSpec. (8.11)
```

Compression is useful, but dangerous.

If compression removes boundary, gate, or residual, the Skill becomes unsafe.

```
BadCompression = TokenSavings − Boundary − Gate − Residual. (8.12)
```

A good Skill Spec should say what cannot be compressed away.

8.7 Pipeline Completion

A Skill pipeline should define what counts as completion.

Completion may be:

```
full success;
partial success;
blocked;
refused;
escalated;
failed.
```

These should be distinct.

A Skill should not treat partial success as full success.

Example:

```
Full success: all required artifacts inspected, output generated, validation passed, residual disclosed.

Partial success: output generated but validation tool unavailable.

Blocked: required artifact missing and user clarification needed.

Refused: task outside authority or violates safety rule.

Escalated: human judgment required.

Failed: runtime or logic error prevents meaningful output.
```

Formula:

```
PipelineOutcome ∈ {Success, PartialSuccess, Blocked, Refused, Escalated, Failed}. (8.13)
```

This prevents false completion.

---

9. Gate Stack

A gate decides when a stage, claim, output, or action is allowed to become accepted.

In a Skill Spec, gates are not optional decorations.

They are the core control mechanism.

A gate turns raw possibility into committed progress.

```
Gate = RecognitionRule + CommitmentThreshold + AuthorityCondition. (9.1)
```

In Philosophical Interface Engineering, a gate is the rule by which something becomes recognized inside a declared system. A raw occurrence becomes an event only after passing a gate, and the interface formula includes boundary, observables, gate, trace, residual, invariance, and revision as the essential structure of a usable world.

For Agent Skills, the same principle becomes practical.

A Skill should not simply "continue."

It should pass gates.

9.1 Why Gates Matter

Without gates, the Skill over-commits.

It may:

```
answer before reading required files;
generate code before understanding source behavior;
cite weak evidence as strong evidence;
treat assumptions as facts;
perform external actions without approval;
declare success without validation;
hide missing information.
```

Gate failure creates false closure.

```
FalseClosure = Output − ValidGate − ResidualHonesty. (9.2)
```

A strong gate system prevents this.

It forces the Skill to ask:

```
Do I have enough input?
Is the user intent clear?
Is this tool allowed?
Is this evidence sufficient?
Is this output validated?
Is human approval required?
What residual remains?
```

Gate discipline is therefore one of the main differences between an ordinary prompt and a serious Skill.

9.2 The Gate Stack

A complex Skill should have multiple gates.

The standard gate stack is:

```
SkillGateStack = EntryGate + ClarificationGate + ArtifactGate + ToolGate + EvidenceGate + RiskGate + ValidationGate + HumanGate + ExitGate. (9.3)
```

Each gate controls a different kind of commitment.

Gate	Question
Entry Gate	Should this Skill run?
Clarification Gate	Is the user request clear enough?
Artifact Gate	Are required inputs available?
Tool Gate	Are required tools available and allowed?
Evidence Gate	Is support sufficient for the claim?
Risk Gate	Does this require safety limitation or escalation?
Validation Gate	Does the output satisfy the contract?
Human Gate	Must a person approve before action?
Exit Gate	Is the run complete, partial, blocked, refused, or escalated?

This gate stack should be tailored to the Skill.

A low-risk formatting Skill may need only entry and output gates.

A code migration Skill may need artifact, tool, validation, and residual gates.

A legal or financial Skill may need risk and human gates.

9.3 Entry Gate

The Entry Gate decides whether the Skill should run at all.

A Skill should run only when the task matches its declared purpose.

Example:

```
EntryGate(Task) = Run if TaskClass ∈ SupportedTaskClasses and RiskClass ≤ SkillAuthority. (9.4)
```

If the task does not fit, the Skill should not force itself.

Possible outcomes:

```
Run Skill;
Use Simpler Prompt;
Ask Clarification;
Route to Another Skill;
Refuse;
Escalate.
```

Formula:

```
EntryGate(Task) ∈ {Run, Simplify, Clarify, Route, Refuse, Escalate}. (9.5)
```

This prevents false rigor.

A complex Skill should not be applied merely because it exists.

9.4 Clarification Gate

The Clarification Gate decides whether user intent is clear enough.

A Skill may ask for clarification when:

```
target output is unclear;
input artifact is missing;
scope is ambiguous;
risk level is uncertain;
platform action is irreversible;
success criteria are undefined;
multiple interpretations are plausible.
```

Formula:

```
ClarificationNeeded ⇔ Ambiguity affects Boundary, Output, Risk, or IrreversibleAction. (9.6)
```

Not every ambiguity requires a question.

Sometimes the Skill can proceed with a stated assumption.

```
MinorAmbiguity → ProceedWithAssumption + ResidualNote. (9.7)
```

But major ambiguity should stop the pipeline.

```
MajorAmbiguity → AskClarification. (9.8)
```

A Skill Spec should define the difference.

9.5 Artifact Gate

The Artifact Gate decides whether required inputs are available.

Example:

```
Do not perform code migration unless source code or source behavior is available.
```

For a document review Skill:

```
Do not summarize a document unless the document has been read or its absence is disclosed.
```

For a spreadsheet Skill:

```
Do not claim workbook-level structure unless sheets have been inspected.
```

Formula:

```
ArtifactGate = RequiredArtifactsAvailable ∨ MissingArtifactsDeclared. (9.9)
```

This formula is important.

Sometimes the Skill can proceed without all artifacts, but only if missing artifacts are declared.

The gate does not always require complete data.

It requires honesty about data completeness.

9.6 Tool Gate

The Tool Gate decides whether tool use is necessary, allowed, and available.

It asks:

```
Is a tool needed?
Is the tool available?
Is the tool allowed under policy?
Does tool use require approval?
What happens if the tool fails?
```

Formula:

```
ToolGate(Tool, Action) = Needed ∧ Available ∧ Authorized ∧ SafeForScope. (9.10)
```

If a required tool is unavailable, the Skill should not pretend the tool result exists.

Possible outcomes:

```
fallback method;
partial output;
ask user;
human escalation;
refusal;
residual disclosure.
```

Example:

```
If code execution is unavailable, static analysis may proceed, but output must be marked unverified.
```

This gate is essential for portability.

Different platforms expose different tools.

The Tool Gate protects the invariant core from platform variation.

9.7 Evidence Gate

The Evidence Gate controls claims.

It asks:

```
Is this claim supported by inspected evidence?
Is the evidence direct or inferred?
Is citation required?
Is the source authoritative enough?
Is there conflicting evidence?
Is the claim time-sensitive?
```

Formula:

```
EvidenceGate(Claim) = SupportLevel ≥ RequiredSupportLevel(ClaimType, RiskLevel). (9.11)
```

Claim types may include:

```
source fact;
inference;
recommendation;
speculation;
warning;
decision;
verified result.
```

The Skill Spec should define how to label each type.

Example:

```
A source fact may be stated if directly supported.
An inference must be marked as inference.
A recommendation must disclose assumptions.
A verified result requires validation evidence.
A regulated decision must remain with the user or qualified professional.
```

Without evidence gates, AI output becomes fluent but unsafe.

9.8 Risk Gate

The Risk Gate controls high-impact decisions.

It asks:

```
Could this output affect legal, financial, medical, safety, employment, privacy, security, or external-system outcomes?
```

If yes, stronger rules apply.

Formula:

```
RiskGate(Action) = Allow if RiskLevel ≤ SkillAuthority and RequiredSafeguards satisfied. (9.12)
```

Risk levels may be:

```
R0 = low-risk formatting or explanation;
R1 = ordinary drafting or analysis;
R2 = professional judgment support;
R3 = high-impact recommendation;
R4 = irreversible external action;
R5 = prohibited or unsafe action.
```

Formula:

```
RiskLevel ∈ {R0, R1, R2, R3, R4, R5}. (9.13)
```

Each Skill should declare its maximum authority.

```
SkillAuthority = MaxRiskLevel the Skill may handle without escalation. (9.14)
```

For many Skills:

```
SkillAuthority ≤ R2. (9.15)
```

That means the Skill can support analysis but cannot own final regulated judgment.

9.9 Validation Gate

The Validation Gate checks whether the output satisfies the output contract.

Validation may be:

```
schema validation;
format validation;
test execution;
source comparison;
cross-checking;
human review;
citation audit;
logic consistency check;
risk checklist;
regression test.
```

Formula:

```
ValidationGate(Output) = ContractSatisfied ∧ NoCriticalResidualHidden. (9.16)
```

This is stronger than "the answer looks good."

For a code Skill:

```
Validation may require compilation or tests.
```

For a document Skill:

```
Validation may require every major claim to be linked to source evidence.
```

For a spreadsheet Skill:

```
Validation may require workbook formulas, pivot sources, and output values to be checked.
```

For an article-writing Skill:

```
Validation may require topic coverage, style requirements, formula format, and section continuity.
```

A Skill that cannot validate should say so.

```
NoValidationAvailable → OutputStatus = Draft or Unverified. (9.17)
```

9.10 Human Gate

The Human Gate controls actions or judgments that must remain with a person.

Examples:

```
send this email;
delete these files;
publish this article;
submit this application;
approve this legal position;
make this investment;
diagnose this illness;
deploy this code;
store this memory;
change this account setting.
```

Formula:

```
HumanGateRequired ⇔ Action is External ∨ Irreversible ∨ Regulated ∨ Identity-Binding ∨ Sensitive. (9.18)
```

The Skill Spec should state whether the Skill may:

```
recommend;
draft;
prepare for review;
execute after approval;
never execute.
```

This distinction is crucial.

```
Draft ≠ Execute. (9.19)
```

```
Recommend ≠ Decide. (9.20)
```

```
Prepare ≠ Commit. (9.21)
```

Many AI safety failures come from collapsing these distinctions.

9.11 Exit Gate

The Exit Gate decides the final status of the Skill run.

It should not always say "done."

Possible statuses:

```
Success;
PartialSuccess;
Blocked;
NeedsUserInput;
NeedsHumanApproval;
Escalated;
Refused;
Failed.
```

Formula:

```
ExitStatus = Evaluate(Gates, OutputContract, Residual, Risk). (9.22)
```

Example:

```
Success: output contract satisfied, validation passed, residual disclosed.
```

```
PartialSuccess: useful output produced, but some validation or artifact condition missing.
```

```
Blocked: required information missing.
```

```
NeedsHumanApproval: action cannot be committed by AI alone.
```

```
Refused: task violates boundary or safety condition.
```

```
Failed: runtime error or insufficient capability prevents useful completion.
```

This makes closure honest.

9.12 Gate Failure Modes

Gates can fail in predictable ways.

1. Missing Gate

The Skill has no rule for commitment.

Example:

```
The Skill generates final output without checking whether the user supplied required data.
```

2. Loose Gate

The Skill accepts weak evidence.

Example:

```
A legal summary treats an uncertain interpretation as settled law.
```

3. Rigid Gate

The Skill refuses useful partial work because full evidence is unavailable.

Example:

```
A document Skill refuses to summarize a partial file even though a partial summary with residual would help.
```

4. Hidden Gate

The Skill uses implicit criteria not declared in the specification.

Example:

```
The agent decides what counts as "important" without saying its selection rule.
```

5. Captured Gate

The Skill lets user pressure override validation or safety.

Example:

```
The user says "just say it is verified," and the Skill complies without evidence.
```

6. Misplaced Gate

The Skill validates the wrong thing.

Example:

```
A code Skill checks style but not behavior.
```

Formula:

```
GateFailure = MissingGate ∨ LooseGate ∨ RigidGate ∨ HiddenGate ∨ CapturedGate ∨ MisplacedGate. (9.23)
```

A mature Skill Spec should list likely gate failures and remedies.

9.13 Gate Design Best Practices

A strong gate should be:

```
explicit;
stage-specific;
risk-adjusted;
evidence-sensitive;
auditable;
residual-aware;
human-escalatable;
portable.
```

Formula:

```
GoodGate = Explicit + EvidenceBound + RiskAdjusted + Auditable + ResidualHonest. (9.24)
```

A gate should not merely block.

It should create governed commitment.

The best gates allow the Skill to proceed where appropriate, but force it to disclose uncertainty where closure is incomplete.

```
GoodGate ≠ AlwaysBlock. (9.25)
```

```
GoodGate = ResponsibleCommitment. (9.26)
```

---

10. Tool and Resource Contract

A Skill often depends on tools.

Tools may include:

```
file search;
web search;
code execution;
spreadsheet processing;
PDF rendering;
calendar access;
email access;
database query;
API call;
browser automation;
image generation;
document creation;
test runner;
version control;
retrieval system.
```

But tool availability differs across platforms.

Therefore, a Skill Spec must define a Tool and Resource Contract.

```
ToolContract = RequiredTools + OptionalTools + ForbiddenTools + ToolGate + FallbackRules + ToolTrace. (10.1)
```

10.1 Required Tools

Required tools are tools without which the Skill cannot perform its core function.

Example:

```
A spreadsheet editing Skill requires spreadsheet read/write capability.
```

```
A code execution validation Skill requires code execution or test runner access.
```

```
A current-events research Skill requires web search.
```

The Skill Spec should identify these clearly.

If a required tool is unavailable, the Skill should not pretend to complete the task.

```
MissingRequiredTool → Blocked or PartialOutputWithResidual. (10.2)
```

10.2 Optional Tools

Optional tools improve quality but are not strictly required.

Example:

```
A document summary Skill may work from text alone, but PDF rendering improves analysis of charts and visual layout.
```

```
A coding Skill may produce static suggestions without running tests, but code execution improves validation.
```

```
A research Skill may use uploaded sources only, while web search improves recency.
```

Optional tools should have fallback rules.

```
OptionalToolUnavailable → Continue + RecordReducedConfidence. (10.3)
```

10.3 Forbidden Tools

Some tools should not be used.

A Skill Spec may forbid tools because of:

```
privacy;
security;
cost;
legal restriction;
user instruction;
platform policy;
risk of external action;
unreliable source;
scope mismatch.
```

Example:

```
This Skill must not send emails. It may draft emails only.
```

```
This Skill must not delete or overwrite files.
```

```
This Skill must not use web search when the user has requested analysis only of uploaded documents.
```

Formula:

```
ForbiddenToolUse = ToolAction outside Boundary or Authority. (10.4)
```

10.4 Tool Fallback Rules

Fallback rules define what to do when a tool is unavailable.

Possible fallback types:

```
manual fallback;
static reasoning fallback;
ask-user fallback;
reduced-output fallback;
residual-only fallback;
refusal fallback;
escalation fallback.
```

Example:

```
If test runner is unavailable, perform static review and mark behavioral equivalence as unverified.
```

Example:

```
If file search is unavailable, ask the user to provide relevant excerpts.
```

Example:

```
If web search is unavailable for a current-fact question, state that current verification cannot be performed.
```

Formula:

```
Fallback = AlternativePath + CapabilityReduction + ResidualDisclosure. (10.5)
```

A fallback is not complete unless it discloses the capability reduction.

10.5 Tool Trace

Every significant tool use should leave trace.

A Tool Trace may include:

```
tool name;
purpose;
input summary;
output summary;
success or failure;
error message;
impact on final output;
residual created.
```

Formula:

```
ToolTrace = ToolName + Purpose + Input + Output + Error + OutputImpact + Residual. (10.6)
```

This does not mean exposing private internal logs to every user.

It means the Skill Spec should define what trace is needed for audit, debugging, or user trust.

For high-risk Skills, tool trace may need to be structured and retained.

For low-risk Skills, a user-facing summary may be enough.

10.6 Resource Contract

Resources are not always tools.

A resource may be:

```
document;
schema;
template;
style guide;
policy file;
reference table;
example set;
test fixture;
rubric;
memory entry;
retrieval corpus;
API documentation.
```

A Skill Spec should define resource requirements.

Example:

```
This Skill requires a style guide resource when producing organization-specific documents.
```

Example:

```
This Skill requires golden test fixtures when claiming migration equivalence.
```

Formula:

```
ResourceContract = RequiredResources + OptionalResources + SourcePriority + FreshnessRule + ValidationRule. (10.7)
```

Source priority is especially important.

The Skill Spec should say whether uploaded files, user instructions, official documentation, or web sources are authoritative.

```
SourcePriority = UserDeclaredSource > ProjectSource > OfficialSource > ExternalSource > ModelMemory. (10.8)
```

This priority may vary by Skill.

But it should be declared.

10.7 Tool Authority

Tools differ in authority.

Some tools only read.

Some write.

Some modify external systems.

Some communicate with other people.

Some spend money.

Some change user settings.

Therefore, the Skill Spec should classify tools by authority level.

```
ToolAuthority ∈ {ReadOnly, GenerateArtifact, ModifyLocal, ModifyExternal, Communicate, Transact, Govern}. (10.9)
```

Examples:

```
ReadOnly: file search, document reading.
GenerateArtifact: create report, create draft file.
ModifyLocal: edit local project files.
ModifyExternal: update calendar, send message, change database.
Communicate: send email, post message.
Transact: purchase, transfer, subscribe.
Govern: change permissions, policies, memory, or account settings.
```

Higher authority requires stronger gates.

```
RequiredGateStrength ∝ ToolAuthority. (10.10)
```

This should be explicit.

---

11. State and Memory Policy

State and memory are central to Agent Skills.

They are also common sources of failure.

A Skill may assume that the agent remembers previous decisions.

But the platform may not.

A Skill may assume that a correction will affect future behavior.

But the runtime may only store a passive log.

A Skill may assume that a user preference should persist.

But privacy or safety rules may forbid storing it.

Therefore, every Skill Spec should include a State and Memory Policy.

```
StateMemoryPolicy = RuntimeState + SessionMemory + PersistentMemory + MemoryGate + ForgetRule + TraceRule. (11.1)
```

11.1 State versus Memory

State and memory are related but different.

State is the current working condition of a task.

Memory is what persists across time.

```
State = Current Working Configuration of a Skill Run. (11.2)
```

```
Memory = Information Retained beyond Immediate Working Context. (11.3)
```

Example state:

```
The Skill is currently in validation stage.
```

Example memory:

```
The user prefers Blogger-ready Unicode formula formatting.
```

Example trace:

```
A prior verifier failure causes the Skill to require stronger evidence for similar future outputs.
```

Memory stores.

Trace changes future behavior.

The PIE glossary makes the same distinction between a passive log and active trace: a log stores information, while trace bends future projection or behavior.

For Agent Skills, this distinction is critical.

A Skill should not claim to have “learned” if it merely stored a log that never affects future execution.

11.2 Stateless Skills

A stateless Skill treats each run independently.

Example:

```
Summarize this paragraph.
```

No persistent memory is required.

Formula:

```
StatelessSkill(Run_k) independent of Run_(k-1). (11.4)
```

Stateless Skills are easier to port.

They are safer.

They are simpler to test.

But they cannot accumulate user preferences, project context, or correction history unless those are provided again as input.

11.3 Session-State Skills

A session-state Skill remembers information within one task or conversation.

Example:

```
The user uploaded three files. The Skill remembers which files have already been inspected during this conversation.
```

Formula:

```
SessionState = State retained within current interaction boundary. (11.5)
```

This is useful for multi-turn work.

But session state should not be assumed to persist beyond the task unless declared.

A Skill Spec should say:

```
This state is valid only inside the current run.
```

11.4 Persistent-Memory Skills

Persistent-memory Skills use information across sessions.

Examples:

```
user style preferences;
project conventions;
known repository structure;
repeated correction history;
approved workflow settings;
long-term research agenda.
```

Persistent memory is powerful but risky.

It raises questions:

```
What may be stored?
Who approved storage?
Can the user inspect it?
Can it be corrected?
Can it be forgotten?
Does it contain sensitive data?
Does it affect future outputs?
```

Formula:

```
PersistentMemoryWrite = UserPermission + FutureUseJustification + SensitivityCheck + ForgetPath. (11.6)
```

A Skill should never silently persist important user information unless the runtime policy allows it and the user's consent is clear.

11.5 Memory Write Gate

A Memory Write Gate decides whether information should be stored.

It asks:

```
Is the information stable?
Is it useful for future tasks?
Is it non-sensitive or explicitly approved?
Is it user-provided or reliably inferred?
Is the user likely to expect future use?
Can it be corrected or forgotten?
```

Formula:

```
MemoryWriteGate(Item) = Stable ∧ Useful ∧ Permitted ∧ Correctable ∧ NonSensitiveUnlessApproved. (11.7)
```

Examples of good memory candidates:

```
The user prefers article math in Blogger-ready Unicode style.
The user wants technical documents before implementation.
The project uses a specific directory structure.
The team requires residual audit in generated specs.
```

Examples of poor memory candidates:

```
temporary mood;
one-off task detail;
sensitive personal fact;
unverified inference;
short-lived preference;
private data not needed later.
```

Memory policy should be explicit.

11.6 Trace Memory

Trace memory is different from ordinary memory.

It records events that should change future behavior.

Examples:

```
A previous generated SQL failed equivalence comparison.
A previous document summary missed embedded chart text.
A previous Skill run used an unsafe assumption.
A user corrected a mapping rule.
A test case exposed a recurring edge case.
```

Trace memory should affect future gates.

Formula:

```
TraceMemory_(k+1) = TraceMemory_k ⊔ VerifiedCorrection_k. (11.8)
```

The symbol ⊔ means the new trace is joined into the prior trace without pretending it erases the past.

For Skills, trace memory supports improvement.

```
NoTrace → RepeatedFailure. (11.9)
```

But trace memory must be governed.

Too much trace may make the Skill rigid.

Too little trace makes it repeat mistakes.

```
HealthyTrace = EnoughMemoryToLearn − ExcessMemoryThatFreezesAdaptation. (11.10)
```

11.7 Forgetting and Revision

A mature memory policy includes forgetting.

Forgetting may be required when:

```
the user requests deletion;
the memory is wrong;
the memory is outdated;
the memory is sensitive;
the memory was written without proper authority;
the project context changed;
the Skill Spec version changed.
```

Formula:

```
ForgetRule(Item) = UserRequest ∨ Incorrect ∨ Outdated ∨ Unauthorized ∨ Unsafe. (11.11)
```

Forgetting should also leave an audit trace when appropriate.

A system may need to record:

```
A memory was removed at user request.
```

without preserving the forbidden content itself.

This distinction is important:

```
ForgetContent ≠ ForgetThatForgettingOccurred. (11.12)
```

In high-governance settings, the fact of revision may matter.

11.8 State and Memory Failure Modes

Common state and memory failures include:

1. Phantom Memory

The Skill assumes it remembers something, but the runtime does not.

```
PhantomMemory = Skill depends on unavailable prior context. (11.13)
```

2. Silent Persistence

The Skill stores information without user expectation or approval.

```
SilentPersistence = MemoryWrite without declared gate. (11.14)
```

3. Stale Memory

The Skill uses outdated information.

```
StaleMemory = PastTrace treated as current truth. (11.15)
```

4. Overfitted Memory

The Skill over-applies one past correction to unrelated cases.

```
OverfittedMemory = LocalCorrection treated as universal rule. (11.16)
```

5. Missing Trace

The Skill repeats a known mistake because the correction was not retained.

```
MissingTrace = VerifiedFailure not converted into future gate change. (11.17)
```

6. Memory-Trace Confusion

The Skill stores information but does not know whether it should change future behavior.

```
MemoryTraceConfusion = StoredData without GovernanceRole. (11.18)
```

A good Skill Spec should warn against these failures.

Closing of Part III

Part III has moved the Skill Spec from static declaration into runtime structure.

A portable Skill must define:

```
pipeline;
stage contracts;
gate stack;
tool contract;
resource contract;
state policy;
memory policy.
```

The key principles are:

```
A Skill is a governed pipeline, not a persona. (11.19)
```

```
A gate is responsible commitment, not mere blocking. (11.20)
```

```
A tool is not neutral; it carries authority and must leave trace. (11.21)
```

```
Memory is not automatically trace; trace is memory that changes future execution. (11.22)
```

The next part turns to trust:

```
Trace Design, Residual Audit, Output Contract, and Evaluation.
```

Part IV — Trace, Residual, Output Contract, and Evaluation

12. Trace Design

A complex Agent Skill should not merely produce an output.

It should leave trace.

Trace is the record of execution that helps a user, developer, auditor, or future agent understand what happened, what was trusted, what was rejected, what failed, and what remains unresolved.

A trace is not merely a log.

A log records.

A trace governs future interpretation.

```
Log = Stored Record. (12.1)
```

```
Trace = Stored Record that Changes Future Interpretation, Routing, Validation, or Revision. (12.2)
```

For Agent Skills, trace is what makes the difference between a fluent answer and an auditable execution.

A fluent answer may be useful.

But a traced answer can be inspected, corrected, ported, tested, and improved.

```
SkillTrust = OutputQuality + TraceQuality + ResidualHonesty. (12.3)
```

12.1 Why Trace Matters

A Skill without trace may still produce a good answer.

But when it fails, the failure is hard to diagnose.

Suppose a code migration Skill produces incorrect output.

Without trace, we may not know whether the error came from:

```
misread source file;
wrong business-rule interpretation;
tool failure;
missing test case;
bad adapter;
model hallucination;
validation skipped;
human approval missing;
residual hidden.
```

With trace, the team can locate the failure.

```
FailureDiagnosis = Compare(Output, Trace, Gates, Residual). (12.4)
```

Trace therefore supports:

```
debugging;
user trust;
regression testing;
cross-platform porting;
governance;
safety review;
future improvement;
human accountability.
```

A Skill that cannot be traced cannot mature.

12.2 Trace as Artifact Lineage

A Skill trace should record how outputs were produced from inputs.

Example:

```
User Request
→ Input Files
→ Artifact Inventory
→ Parsed Structure
→ Intermediate Mapping
→ Generated Draft
→ Validation Result
→ Final Output
→ Residual Register
```

This is artifact lineage.

```
ArtifactLineage = Input → IntermediateArtifact → Validation → Output → Residual. (12.5)
```

Artifact lineage is especially important when the Skill transforms material.

Examples:

```
legacy code → new code;
contract document → risk report;
spreadsheet → pivot summary;
source article → evidence map;
user requirement → technical specification;
theoretical article → runtime kernel.
```

Without lineage, the output floats.

With lineage, the output can be traced back to source.

12.3 Trace as Decision Record

A Skill also makes decisions.

Examples:

```
which files were relevant;
which tool was used;
which interpretation was selected;
which branch of the pipeline was chosen;
which claim was treated as supported;
which missing input was tolerated;
which residual was carried forward.
```

These decisions should be recorded when they matter.

```
DecisionTrace = Decision + Reason + Evidence + Gate + Residual. (12.6)
```

Example:

```
Decision: Proceed with static analysis rather than execution.
Reason: Test runner unavailable.
Evidence: Runtime did not provide code execution.
Gate: Tool Gate failed for execution; fallback allowed.
Residual: Behavioral equivalence remains unverified.
```

This kind of trace prevents false confidence.

12.4 Trace as Gate Record

A Skill should record which gates passed, failed, or were bypassed.

Example gate record:

```
Entry Gate: passed.
```

```
Artifact Gate: partial pass; two required files missing.
Tool Gate: failed for code execution; static fallback used.
Evidence Gate: passed for source facts; inference marked.
Validation Gate: partial; schema valid but behavior untested.
Human Gate: not required.
Exit Gate: PartialSuccess.
```

Formula:

```
GateTrace = {GateName, Status, Reason, Evidence, Residual}. (12.7)
```

Gate Trace is useful because many AI failures are not content failures.

They are gate failures.

The answer may be grammatically perfect, but the wrong gate was passed.

12.5 Trace Levels

Not all Skills need the same trace depth.

A Skill Spec should define trace level.

Possible levels:

```
T0 = No retained trace.
T1 = User-visible summary only.
T2 = Internal debug log.
T3 = Structured audit trail.
T4 = Compliance-grade trace.
T5 = Reproducible execution ledger.
```

Formula:

```
TraceLevel ∈ {T0, T1, T2, T3, T4, T5}. (12.8)
```

Suggested use:

Trace Level	Use Case
T0	Low-risk one-off transformation
T1	Ordinary user-facing task
T2	Developer debugging
T3	Reusable team Skill
T4	Enterprise, regulated, or high-stakes workflow
T5	Scientific, legal, safety, or reproducibility-critical workflow

Trace depth should increase with risk.

```
RequiredTraceLevel ∝ RiskLevel + ReproducibilityNeed + AuditNeed. (12.9)
```

12.6 User-Facing Trace versus Internal Trace

Not every trace should be shown to the user.

A Skill may maintain internal trace for debugging while showing a concise user-facing summary.

User-facing trace may include:

```
what sources were used;
what assumptions were made;
what could not be verified;
what output status applies;
what residual remains.
```

Internal trace may include:

```
stage transitions;
tool input summaries;
tool output summaries;
validation details;
error diagnostics;
adapter behavior;
retry history.
```

Formula:

```
Trace = UserFacingTrace + InternalExecutionTrace. (12.10)
```

The Skill Spec should define what belongs in each.

For many tasks, the user does not need all internal details.

But the user does need enough trace to understand trust level.

12.7 Privacy and Trace

Trace can create privacy risk.

A Skill should not retain more than necessary.

Trace policy should answer:

```
What trace is required?
Who can inspect it?
How long is it retained?
Can the user delete it?
Does it contain sensitive data?
Can it be anonymized?
Is the trace necessary for safety or audit?
```

Formula:

```
SafeTrace = NecessaryTrace − UnneededSensitiveRetention. (12.11)
```

Trace is valuable.

But trace without governance becomes surveillance.

Therefore, a mature Skill Spec balances auditability and privacy.

---

13. Residual Audit

Residual is what remains unresolved after the Skill has done what it can responsibly do.

Residual may be missing information, ambiguity, unverified claims, platform limitation, tool failure, conflicting evidence, incomplete validation, or human judgment still required.

```
Residual = Unfinished Material after Governed Closure. (13.1)
```

A Skill without residual audit tends to over-close.

It presents a clean answer even when uncertainty remains.

This may look helpful in the short term.

But it weakens trust.

```
FalseConfidence = FluentOutput − ResidualDisclosure. (13.2)
```

Residual Audit prevents false confidence.

13.1 Residual Is Not Failure

Residual does not mean the Skill failed.

Residual often means the Skill remained honest.

Example:

```
The source file is missing.
```

This is residual.

Example:

```
The claim is plausible but not verified.
```

This is residual.

Example:

```
The code compiles, but behavioral equivalence was not tested.
```

This is residual.

Example:

```
The legal interpretation depends on jurisdiction and should be reviewed by a qualified professional.
```

This is residual.

A mature Skill does not hide these.

```
MatureClosure = Output + ResidualHonesty. (13.3)
```

13.2 Residual Types

A Skill Spec should define residual categories.

Common categories:

```
MissingData;
AmbiguousIntent;
UnsupportedInference;
ConflictingEvidence;
ToolUnavailable;
ToolFailed;
ValidationIncomplete;
PlatformLimitation;
HumanJudgmentRequired;
SafetyBoundaryReached;
```

```
OutOfScopeMaterial;
TimeSensitiveUnverifiedFact;
CompressionLoss;
AdapterMismatch;
MemoryUncertainty.
```

Formula:

```
ResidualType ∈ {MissingData, Ambiguity, Conflict, ToolLimit, ValidationGap, HumanJudgment, SafetyLimit, ScopeLimit, TimeLimit, CompressionLoss, AdapterMismatch, MemoryUncertainty}. (13.4)
```

This taxonomy lets the Skill describe residual precisely.

A vague statement like:

```
There may be limitations.
```

is weak.

A stronger residual statement says:

```
ValidationIncomplete: code execution was unavailable, so behavioral equivalence was not verified.
```

Specific residual is useful.

Vague residual is decoration.

13.3 Residual Register

A residual register is a structured list of unresolved items.

Example:

ID	Residual Type	Description	Impact	Suggested Action
R1	MissingData	Source file `query_2.sql` unavailable	Cannot verify full migration	Ask user to provide file
R2	ToolUnavailable	Test runner unavailable	Output is static-only	Run tests manually
R3	UnsupportedInference	Business rule inferred from naming	May be wrong	Confirm with domain user
R4	ValidationIncomplete	Generated output not compared to legacy output	Equivalence unverified	Perform comparison

Formula:

```
ResidualRegister = {ResidualID, Type, Description, Impact, Action}. (13.5)
```

A residual register is better than a vague caveat because it gives the next action.

Residual should not only warn.

It should guide revision.

13.4 Residual Severity

Not all residual has equal importance.

A Skill Spec should classify residual severity.

Possible levels:

```
S0 = informational;
S1 = minor uncertainty;
S2 = moderate limitation;
S3 = major limitation;
S4 = blocking issue;
S5 = safety-critical issue.
```

Formula:

```
ResidualSeverity ∈ {S0, S1, S2, S3, S4, S5}. (13.6)
```

Example:

```
S1: optional formatting preference unknown.
S2: one supporting source missing.
S3: major assumption unverified.
S4: required artifact missing.
S5: unsafe or regulated action cannot proceed.
```

Exit status should depend on residual severity.

```
If max(ResidualSeverity) ≥ S4, ExitStatus ≠ Success. (13.7)
```

This prevents the Skill from claiming full completion when blocking residual remains.

13.5 Residual Ownership

Residual should have an owner.

Possible owners:

```
user;
AI Skill;
human reviewer;
platform adapter;
external tool;
domain expert;
future run;
project team.
```

Example:

```
Missing requirement clarification → user.
Code behavior verification → test runner or developer.
Legal interpretation → qualified professional.
Adapter mismatch → platform engineer.
Output revision → AI Skill.
```

Formula:

```
ResidualOwner = Party responsible for resolving or accepting residual. (13.8)
```

Without ownership, residual becomes noise.

With ownership, residual becomes action.

13.6 Residual as Revision Pressure

Residual should feed future improvement.

A recurring residual may indicate a Skill weakness.

Example:

```
If many runs produce MissingData residual, improve artifact intake.
```

Example:

```
If many runs produce ValidationIncomplete residual, add a validation tool.
```

Example:

```
If many runs produce AmbiguousIntent residual, improve clarification prompts.
```

Formula:

```
RepeatedResidual → SkillRevisionCandidate. (13.9)
```

Residual is not just leftover.

Residual is the future of the Skill asking to be redesigned.

13.7 Residual Footer

For user-facing outputs, a residual footer is often useful.

Example:

```
Residual / Open Issues:
1. I could not verify the generated code by running tests in this environment.
2. The mapping for field X is inferred from naming and should be confirmed.
3. The final deployment decision requires human review.
```

A residual footer should be:

```
specific;
short;
actionable;
severity-aware;
not alarmist;
not hidden.
```

Formula:

```
ResidualFooter = TopOpenIssues + TrustImpact + SuggestedNextStep. (13.10)
```

A good residual footer helps the user know what to trust and what not to trust.

---

14. Output Contract

A Skill must define what its output should look like.

Without an output contract, the same Skill may generate inconsistent deliverables across platforms and runs.

```
OutputContract = RequiredFields + OptionalFields + Format + ErrorShape + PartialShape + ResidualFooter + ValidationRules. (14.1)
```

An output contract is not merely formatting.

It is part of the Skill's gate system.

The Skill is not complete until its output satisfies the contract or honestly declares why it cannot.

14.1 Why Output Contracts Matter

AI systems are naturally flexible.

This flexibility is useful, but it can reduce reproducibility.

A Skill may produce:

```
a narrative answer in one run;
a table in another run;
a JSON object in another run;
a long report in another run;
a short bullet list in another run.
```

This may be acceptable for casual use.

It is not acceptable for reusable Skills that need integration, review, automation, or regression testing.

Output contracts create consistency.

```
ConsistentOutput = SkillOutput constrained by OutputContract. (14.2)
```

14.2 Human-Readable Output

Human-readable output is meant for people.

Examples:

```
executive summary;
technical explanation;
risk report;
migration notes;
recommendation memo;
step-by-step instructions;
review comments;
article draft.
```

A human-readable output contract should define:

```
sections;
order;
level of detail;
tone;
citation requirements;
residual placement;
actionability;
audience.
```

Example contract:

```
Output must include:
1. Executive Summary
2. Key Findings
3. Evidence or Source Basis
4. Risks / Limitations
5. Recommended Next Actions
6. Residual / Open Issues
```

This is useful for review-oriented Skills.

14.3 Machine-Readable Output

Machine-readable output is meant for software systems.

Examples:

```
JSON;
YAML;
CSV;
XML;
database row;
schema object;
test result object;
configuration file.
```

Machine-readable output requires stricter schema.

Example:

```
{
  "status": "partial_success",
  "summary": "...",
  "artifacts_read": [],
  "findings": [],
  "residual": [],
  "next_actions": []
}
```

Formula:

```
MachineOutput = Schema + FieldTypes + RequiredKeys + AllowedValues + ValidationRule. (14.3)
```

Machine-readable outputs should avoid vague prose in fields where structured values are expected.

14.4 Dual Output

Many complex Skills should produce dual output:

```
human-readable report;
machine-readable audit object.
```

Formula:

```
DualOutput = HumanSummary + StructuredResult. (14.4)
```

Example:

```
The user sees a clear report.
The system stores a structured object containing status, artifacts, gates, residual, and validation results.
```

This is often the best pattern for enterprise Skills.

Humans get readability.

Systems get consistency.

14.5 Partial Output Contract

A Skill should define partial output.

Partial output is needed when the Skill can provide useful work but cannot fully complete the task.

Example:

```
Status: PartialSuccess
Reason: Required validation tool unavailable.
Completed: Static analysis and draft migration.
Not completed: Runtime equivalence test.
Residual: Behavioral verification required.
Next action: Run provided test script in target environment.
```

Formula:

```
PartialOutput = CompletedWork + MissingWork + Reason + Residual + NextStep. (14.5)
```

This is much better than either refusing everything or pretending full success.

Partial output is honest closure.

14.6 Error Output Contract

A Skill should define how to report errors.

Error output should include:

```
error type;
where it occurred;
whether the task can continue;
what the user can do;
what residual remains.
```

Example:

```
Error Type: Missing Required Artifact
Stage: Artifact Inventory
Impact: Cannot perform migration comparison
User Action: Upload source SQL file
Exit Status: Blocked
```

Formula:

```
ErrorOutput = ErrorType + Stage + Impact + RecoveryAction + ExitStatus. (14.6)
```

Error reporting should be specific.

A generic “something went wrong” is not a Skill-grade failure report.

14.7 Output Status

Every complex Skill output should have a status.

Possible statuses:

```
Success;
PartialSuccess;
Draft;
Unverified;
Blocked;
NeedsUserInput;
NeedsHumanApproval;
Refused;
Failed.
```

Formula:

```
OutputStatus ∈ {Success, PartialSuccess, Draft, Unverified, Blocked, NeedsUserInput, NeedsHumanApproval, Refused, Failed}. (14.7)
```

The status should match the gate results.

A Skill should not label output as “Success” if validation failed.

```
OutputStatus = Function(GateTrace, ValidationResult, ResidualSeverity). (14.8)
```

14.8 Forbidden Output Claims

A Skill Spec should identify forbidden claims.

Examples:

```
Do not say “verified” unless validation was performed.
Do not say “complete” if required artifacts were missing.
Do not say “legally safe” unless qualified legal review occurred.
Do not say “production-ready” unless deployment criteria passed.
Do not say “fact” when the statement is inference.
Do not say “no risk” when residual remains.
```

Formula:

```
ForbiddenClaim = Claim whose RequiredGate has not passed. (14.9)
```

This is one of the most important output-safety rules.

---

15. Evaluation and Test Harness

A Skill cannot be considered mature until it can be tested.

A test harness defines how to check whether the Skill behaves as intended.

```
SkillEvaluation = TestInputs + ExpectedPipeline + ExpectedOutput + ExpectedGateBehavior + ExpectedResidual. (15.1)
```

Testing should not only check final output.

It should check the execution structure.

15.1 Why Tests Are Part of the Skill Spec

Many teams treat evaluation as separate from documentation.

For Agent Skills, this is a mistake.

The Skill Spec defines intended behavior.

The test harness checks whether behavior matches the specification.

```
SkillSpec without Tests = Unverified Contract. (15.2)
```

Tests make the Skill portable because they allow different platform implementations to be compared.

```
CrossPlatformSkillEquivalence = SameTestSet + ComparableGateBehavior + ComparableOutputContract. (15.3)
```

If two implementations pass the same tests, they are meaningfully equivalent even if their internal runtime differs.

15.2 Test Case Structure

Each test case should include:

```
test ID;
test purpose;
input artifacts;
user request;
expected pipeline path;
expected gates;
expected output status;
expected required output fields;
expected residual;
forbidden behavior;
pass/fail criteria.
```

Formula:

```
TestCase = Input + ExpectedPath + ExpectedGates + ExpectedOutput + ExpectedResidual + ForbiddenBehavior. (15.4)
```

Example:

```
Test ID: T-MIG-003
Purpose: Missing source file should trigger artifact residual.
Input: target request but no source file.
Expected Path: Intake → Artifact Gate → NeedsUserInput.
Expected Output Status: Blocked.
Expected Residual: MissingData.
Forbidden Behavior: Do not generate final migration code.
```

This is a useful test because it checks gate discipline, not just content generation.

15.3 Golden Path Test

A golden path test is the normal successful case.

Example:

```
All required inputs are present.
Tools are available.
Risk is low or controlled.
Validation passes.
Output contract is satisfied.
Residual is minor or absent.
```

Formula:

```
GoldenPath = CompleteInput + AvailableTools + ClearIntent + PassedValidation + SuccessOutput. (15.5)
```

Every Skill should have at least one golden path test.

But golden path is not enough.

Most failures happen outside the golden path.

15.4 Ambiguous Input Test

This test checks whether the Skill asks for clarification or proceeds with a declared assumption.

Example:

```
User says: “Analyze this.”
No document type, purpose, or output format is specified.
```

Expected behavior:

```
Ask clarification if purpose affects output.
Proceed with assumption only if ambiguity is minor.
```

Formula:

```
AmbiguityTest = Input with MultiplePlausibleInterpretations. (15.6)
```

This test prevents the Skill from over-interpreting.

15.5 Out-of-Scope Test

This test checks boundary discipline.

Example:

```
A document summary Skill is asked to provide binding legal advice.
```

Expected behavior:

```
Provide general summary if appropriate.
Refuse or disclaim legal judgment.
Escalate to qualified professional.
```

Formula:

```
OutOfScopeTest = Request outside SkillBoundary. (15.7)
```

A Skill that fails this test is unsafe.

15.6 Tool-Unavailable Test

This test checks fallback behavior.

Example:

```
A code Skill is asked to verify behavior, but code execution is unavailable.
```

Expected behavior:

```
Perform static analysis if useful.
Mark behavioral validation as unavailable.
Return PartialSuccess or Unverified, not Success.
```

Formula:

```
ToolUnavailableTest = RequiredToolMissing + ExpectedFallback + ResidualDisclosure. (15.8)
```

This is a key portability test.

15.7 Human Approval Test

This test checks whether the Skill respects human gates.

Example:

```
User asks the Skill to send an email, delete a file, or publish content.
```

Expected behavior:

```
Draft or prepare action.
Request explicit approval before execution.
```

Formula:

```
HumanApprovalTest = ExternalOrIrreversibleAction + ApprovalGateRequired. (15.9)
```

This prevents accidental over-action.

15.8 Unsafe or Regulated Request Test

This test checks risk boundaries.

Examples:

```
medical diagnosis;
legal determination;
investment instruction;
security bypass;
privacy-invasive action;
harmful automation.
```

Expected behavior depends on Skill authority.

Usually:

```
provide safe general information;
refuse unsafe details;
recommend qualified professional;
avoid final decision ownership.
```

Formula:

```
RiskTest = HighImpactRequest + SkillAuthorityCheck + SafeOutput. (15.10)
```

15.9 Partial Success Test

This test checks whether the Skill can produce useful but honest partial output.

Example:

```
Two of three required documents are available.
```

Expected behavior:

```
Analyze available documents.
State missing document.
Do not claim complete review.
Provide residual and next action.
```

Formula:

```
PartialSuccessTest = SomeInputsAvailable + SomeInputsMissing + HonestPartialOutput. (15.11)
```

This prevents all-or-nothing behavior.

15.10 Cross-Platform Equivalence Test

This test checks portability.

The same Skill Spec is implemented in two or more platforms.

Each receives the same test case.

The outputs do not need to be identical word-for-word.

But they should match in:

```
task interpretation;
pipeline path;
gate behavior;
output status;
required fields;
residual disclosure;
forbidden behavior avoidance.
```

Formula:

```
CrossPlatformEquivalence = SameSpec + SameTest + ComparableContractSatisfaction. (15.12)
```

This is the real test of a portable Skill Spec.

If implementations differ wildly, either the Skill Spec is under-specified or one adapter is wrong.

15.11 Regression Test

A regression test ensures that a known previous failure does not return.

Example:

```
Earlier version claimed verification without test execution.
Regression test checks that missing execution now produces Unverified status.
```

Formula:

```
RegressionTest = PreviousFailureCase + ExpectedCorrectedBehavior. (15.13)
```

Regression tests turn failure into trace.

```
VerifiedFailure → RegressionFixture → FutureGate. (15.14)
```

This is how a Skill matures.

15.12 Skill Quality Formula

A mature Skill should be evaluated across several dimensions.

```
SkillQuality = CorrectOutput + StablePipeline + GateDiscipline + ToolHonesty + ResidualHonesty + CrossPlatformConsistency. (15.15)
```

Each term matters.

A Skill with correct output but poor trace is hard to trust.

A Skill with strong pipeline but poor output is not useful.

A Skill with good output but hidden residual is dangerous.

A Skill that works on only one platform may be useful, but not portable.

---

16. Failure Mode Catalogue

Every Skill Spec should include a catalogue of likely failure modes.

Failure modes are not pessimism.

They are engineering maturity.

```
FailureModeCatalogue = KnownWaysThisSkillCanBreak + DetectionSignal + Mitigation. (16.1)
```

A Skill that cannot describe its failure modes has not understood itself.

16.1 Template Overreach

Template overreach occurs when a platform-specific template is treated as universal.

Example:

```
A Skill written for a tool-rich coding agent is reused in a chat-only environment without adaptation.
```

Failure signal:

```
Skill asks to run tools that do not exist.
```

Mitigation:

```
Separate invariant core from adapter layer.
```

Formula:

```
TemplateOverreach = PlatformTemplate treated as CrossPlatformSpec. (16.2)
```

16.2 Adapter Leakage

Adapter leakage occurs when implementation details contaminate the invariant Skill logic.

Example:

```
The Skill assumes all runtimes have shell access because the original implementation used shell scripts.
```

Failure signal:

```
Ported Skill breaks when shell execution is unavailable.
```

Mitigation:

```
Move shell execution into adapter profile; define non-shell fallback.
```

Formula:

```
AdapterLeakage = RuntimeMechanism mistaken for SkillRequirement. (16.3)
```

16.3 Gate Absence

Gate absence occurs when the Skill lacks a commitment rule.

Example:

```
The Skill produces final output without checking whether required evidence exists.
```

Failure signal:

```
Confident answer despite missing source.
```

Mitigation:

```
Add artifact, evidence, and validation gates.
```

Formula:

```
GateAbsence = Commitment without RecognitionRule. (16.4)
```

16.4 Trace Poverty

Trace poverty occurs when the Skill produces output but leaves no useful execution record.

Example:

```
A generated report does not state which documents were inspected or what assumptions were used.
```

Failure signal:

```
Reviewer cannot reproduce or audit the output.
```

Mitigation:

```
Add artifact lineage, decision trace, and residual register.
```

Formula:

```
TracePoverty = Output − ExecutionLineage. (16.5)
```

16.5 Residual Hiding

Residual hiding occurs when the Skill suppresses uncertainty.

Example:

```
The Skill presents an inferred mapping as confirmed.
```

Failure signal:

```
No unresolved issues listed despite incomplete evidence.
```

Mitigation:

```
Require residual footer and severity classification.
```

Formula:

```
ResidualHiding = Closure − OpenIssueDisclosure. (16.6)
```

16.6 Tool Fantasy

Tool fantasy occurs when the Skill assumes unavailable or imaginary tools.

Example:

```
The Skill says it has tested the code, but the runtime had no execution capability.
```

Failure signal:

```
Claimed validation cannot be traced to tool output.
```

Mitigation:

```
Add Tool Gate and Tool Trace.
```

Formula:

```
ToolFantasy = ClaimedToolResult − ActualToolExecution. (16.7)
```

16.7 State Confusion

State confusion occurs when the Skill loses track of current stage, artifacts, or decisions.

Example:

```
The Skill validates an older draft instead of the current output.
```

Failure signal:

```
Trace and final answer refer to different artifacts.
```

Mitigation:

```
Use explicit PipelineState.
```

Formula:

```
StateConfusion = CurrentAction inconsistent with CurrentPipelineState. (16.8)
```

16.8 Memory Misuse

Memory misuse occurs when the Skill uses, writes, or assumes memory incorrectly.

Examples:

```
uses stale user preference;
stores sensitive data unnecessarily;
assumes memory exists when it does not;
fails to remember a verified correction;
over-applies a previous correction.
```

Failure signal:

```
Output depends on unverified or inappropriate prior context.
```

Mitigation:

```
Add Memory Write Gate, Forget Rule, and Trace Memory policy.
```

Formula:

```
MemoryMisuse = StaleMemory ∨ UnauthorizedMemory ∨ PhantomMemory ∨ MissingTrace. (16.9)
```

16.9 Schema Rigidity

Schema rigidity occurs when the output contract is too strict for useful expert judgment.

Example:

```
A legal review Skill forces every issue into a fixed risk score even when qualitative nuance is necessary.
```

Failure signal:

```
Important residual is lost because no schema field allows it.
```

Mitigation:

```
Add free-text residual and expert-comment fields.
```

Formula:

```
SchemaRigidity = StructuredOutput − NecessaryNuance. (16.10)
```

Schemas help.

But schemas must not erase reality.

16.10 False Rigor

False rigor occurs when a complex Skill Spec is used to make weak work look scientific.

Example:

```
A simple brainstorming task is wrapped in excessive gates, pseudo-metrics, and formal output fields.
```

Failure signal:

```
Specification overhead exceeds task value.
```

Mitigation:

```
Use Suitability Gate; choose simpler prompt when appropriate.
```

Formula:

```
FalseRigor = FormalStructure − RealNeed. (16.11)
```

A mature Skill knows when not to run.

16.11 Failure Mode Table

A Skill Spec can include a table like this:

Failure Mode	Detection Signal	Mitigation
Template Overreach	Runtime lacks assumed features	Separate adapter layer

Failure Mode	Detection Signal	Mitigation
Adapter Leakage	Platform detail appears in invariant core	Rewrite as abstract requirement
Gate Absence	Output commits without checks	Add gate stack
Trace Poverty	Reviewer cannot audit output	Add lineage and decision trace
Residual Hiding	Missing uncertainty disclosure	Add residual register
Tool Fantasy	Claimed tool result absent	Add tool trace
State Confusion	Stage/output mismatch	Add explicit state
Memory Misuse	Stale or unauthorized context	Add memory gate
Schema Rigidity	Nuance lost	Add qualitative fields
False Rigor	Over-engineered simple task	Add suitability gate

Formula:

```
SkillFailureRisk = Overreach + HiddenAssumption + MissingGate + LostTrace + ResidualSuppression. (16.12)
```

Closing of Part IV

Part IV has introduced the trust layer of the Technical Skill Specification.

A Skill becomes trustworthy not merely because it gives good answers, but because it defines:

```
trace;
residual;
output contract;
test harness;
failure modes.
```

The central principles are:

```
Trace makes execution inspectable. (16.13)
```

```
Residual makes closure honest. (16.14)
```

```
Output contract makes delivery reproducible. (16.15)
```

```
Tests make portability verifiable. (16.16)
```

```
Failure modes make the Skill self-aware. (16.17)
```

The next part moves from specification to implementation across different platforms:

```
Platform Adapter Mapping and Implementation Profiles.
```

Part V — Platform Adapter Mapping and Implementation Profiles

17. Adapter Mapping Across Agent Platforms

A Technical Skill Specification becomes practical only when it can be mapped into real platforms.

Different agent platforms expose different primitives:

```
instructions;
tools;
files;
resources;
memory;
state;
workflows;
agents;
handoffs;
guardrails;
approval gates;
logs;
schemas;
execution sandboxes.
```

The Skill Spec should not pretend these are identical.

Instead, it should provide an **adapter mapping**.

```
AdapterMapping = Map(SkillSpecConcepts → PlatformPrimitives). (17.1)
```

The invariant Skill remains the same.

The adapter expresses it in the language of the target runtime.

17.1 Why Adapters Are Necessary

A complex Skill cannot assume that every platform has the same execution model.

For example:

```
Platform A may support file search but not code execution.
Platform B may support code execution but not persistent memory.
Platform C may support multi-agent handoff but not direct file editing.
Platform D may support tool calling but require explicit schemas.
Platform E may support local scripts but no cloud tools.
```

If the Skill Spec does not separate core logic from adapter logic, it becomes trapped inside one platform.

```
NoAdapterLayer → PlatformLockIn. (17.2)
```

The adapter layer solves this by saying:

```
Here is the invariant requirement.
Here is how this platform implements it.
Here is what this platform cannot implement.
Here is the residual created by that limitation.
```

This is the essence of honest portability.

17.2 Skill Spec Concepts and Platform Primitives

A useful adapter table can begin like this:

Skill Spec Concept	Platform Mapping Examples
Boundary	system instruction, developer instruction, Skill metadata, task declaration
Input Artifact	uploaded file, repository file, resource, tool result, pasted text
Pipeline Stage	workflow node, graph node, prompt phase, function call, script step
Gate	guardrail, validator, conditional branch, approval prompt, schema check
Tool Contract	tool schema, MCP tool, SDK function, shell script, API wrapper
State	graph state, JSON object, conversation context, database row
Memory	persistent profile, project memory, vector store, trace ledger
Trace	log, span, audit file, report section, database record
Residual	warning, issue list, uncertainty footer, incomplete status
Output Contract	JSON schema, markdown template, file artifact, report structure
Revision	version update, prompt update, Skill folder revision, workflow migration

Formula:

```
PlatformAdapter = ConceptMapping + CapabilityMapping + LimitationMapping + ResidualMapping. (17.3)
```

A strong adapter does not merely say how to implement success.

It also says what happens when the platform lacks a required capability.

17.3 Capability Mapping

Before implementing a Skill on a platform, the adapter should map platform capabilities.

A capability map may include:

```
file_read: yes / no / limited;
file_write: yes / no / approval required;
code_execution: yes / no / sandboxed;
web_search: yes / no / restricted;
persistent_memory: yes / no / user-approved;
tool_calling: yes / no / schema-based;
human_approval: yes / no / manual;
multi_agent: yes / no;
structured_output: yes / no;
logging: none / basic / structured / compliance-grade.
```

Formula:

```
CapabilityMap = {Capability_i → Availability_i + Constraint_i}. (17.4)
```

The adapter then compares the Skill's requirements against the platform's capabilities.

```
AdapterFit = SkillRequirements ∩ PlatformCapabilities. (17.5)
```

If key requirements are missing, the adapter must define fallback or residual.

17.4 Adapter Fit Levels

Not every platform can implement every Skill fully.

A Skill Spec can define adapter fit levels.

```
A0 = Not supported.
A1 = Manual / checklist only.
A2 = Prompt-only implementation.
A3 = Tool-assisted implementation.
A4 = Workflow-orchestrated implementation.
A5 = Fully governed implementation with trace, tests, and approval gates. (17.6)
```

Example:

Fit Level	Meaning
A0	Platform cannot support the Skill safely
A1	Human follows Skill Spec manually
A2	Prompt kernel only; limited trace
A3	Tools available; partial automation
A4	Workflow/state/gates implemented
A5	Full audit, tests, revision, approval, compliance

Formula:

```
AdapterFitLevel = f(CapabilityMatch, RiskSupport, TraceSupport, GateSupport). (17.7)
```

This avoids pretending that a weak platform can provide full Skill behavior.

17.5 Adapter Residual

Every adapter may create residual.

Examples:

```
No code execution → behavioral validation residual.
No file write access → generated artifact must be copied manually.
No persistent memory → prior corrections must be provided each run.
No human approval primitive → approval must be handled outside platform.
No structured logging → audit trace is limited.
No web access → current facts cannot be verified.
```

Formula:

```
AdapterResidual = SkillRequirement − PlatformCapability. (17.8)
```

Adapter residual should be documented.

It is not enough to say:

```
This Skill works on Platform X.
```

A better statement is:

```
This Skill runs on Platform X at Adapter Fit Level A3. Behavioral validation is unavailable, so outputs requiring execution verification must be marked Unverified.
```

This is portable engineering honesty.

17.6 Adapter Mapping Example

Suppose the Skill is:

```
Code Migration Skill.
```

Invariant requirements:

```
read source files;
identify source semantics;
generate target code;
run or define tests;
compare source and target behavior;
produce residual report.
```

Adapter profile for a tool-rich coding platform:

```
read source files through repository access;
execute tests in sandbox;
write generated files;
store trace in project logs;
use structured output for audit.
```

Adapter profile for a chat-only platform:

```
ask user to paste source files;
generate target code as text;
cannot execute tests;
provide manual test instructions;
mark behavioral equivalence as unverified;
include residual footer.
```

Same Skill.

Different adapter.

Different trust level.

```
SameInvariantCore + DifferentAdapter → DifferentExecutionGrade. (17.9)
```

---

18. Implementation Profiles

A Skill Spec may be implemented at different levels of sophistication.

Not every team needs the most complex version.

A good specification should support multiple implementation profiles.

```
ImplementationProfile = SkillSpecDepth + RuntimeCapability + GovernanceNeed. (18.1)
```

The five recommended profiles are:

```
Minimal Profile;
Documented Profile;
Tool-Enabled Profile;
Multi-Agent Profile;
Enterprise-Governed Profile.
```

18.1 Minimal Profile

The Minimal Profile is suitable for individual use, experimentation, or low-risk tasks.

It contains:

```
short purpose statement;
boundary;
basic pipeline;
output format;
residual reminder;
simple checklist.
```

Formula:

```
MinimalProfile = SkillSpec + PromptKernel + Checklist. (18.2)
```

Example use cases:

```
article drafting;
simple document review;
personal workflow;
brainstorming;
non-critical analysis.
```

Advantages:

```
fast;
easy to maintain;
works almost anywhere.
```

Limitations:

```
weak trace;
manual validation;
limited portability testing;
high dependence on user judgment.
```

18.2 Documented Profile

The Documented Profile is suitable for teams.

It contains:

```
full Skill Spec;
examples;
prompt kernel;
expected outputs;
test cases;
failure modes;
porting notes.
```

Formula:

```
DocumentedProfile = SkillSpec + Examples + TestCases + FailureModes. (18.3)
```

Example use cases:

```
team documentation Skill;
repository review Skill;
internal research Skill;
technical writing Skill;
data-cleaning Skill.
```

This profile is often the best starting point.

It does not require deep automation, but it creates shared understanding.

18.3 Tool-Enabled Profile

The Tool-Enabled Profile is suitable when the Skill needs execution support.

It contains:

```
Skill Spec;
tool schemas;
tool gates;
fallback rules;
tool trace;
validation scripts;
structured output.
```

Formula:

```
ToolEnabledProfile = SkillSpec + ToolContract + ValidationTools + ToolTrace. (18.4)
```

Example use cases:

```
code migration;
spreadsheet generation;
PDF analysis;
web research;
```

```
database query;
file transformation.
```

This profile is more powerful, but also riskier.

The Skill must clearly distinguish:

```
tool suggestion;
tool execution;
tool result;
verified conclusion.
```

Formula:

```
ToolResult ≠ VerifiedConclusion unless ValidationGate passes. (18.5)
```

18.4 Multi-Agent Profile

The Multi-Agent Profile is suitable when multiple agents or roles cooperate.

It contains:

```
role boundaries;
handoff gates;
shared artifacts;
shared state;
conflict resolution;
responsibility map;
trace ledger;
final integrator gate.
```

Formula:

```
MultiAgentProfile = RoleSpecs + HandoffRules + SharedLedger + IntegrationGate. (18.6)
```

Example roles:

```
Research Agent;
Implementation Agent;
Reviewer Agent;
Safety Agent;
User-Intent Agent;
Test Agent;
Integrator Agent.
```

The key risk is role confusion.

Each agent must know:

```
what it owns;
what it does not own;
what it may change;
what it must report;
when it must hand off.
```

Formula:

```
RoleBoundary = OwnedArtifacts + AllowedActions + HandoffConditions + ResidualObligations. (18.7)
```

Without role boundaries, multi-agent systems become noisy committees.

18.5 Enterprise-Governed Profile

The Enterprise-Governed Profile is for high-risk, regulated, or production use.

It contains:

```
full Skill Spec;
policy gates;
human approval;
identity and access control;
audit trail;
regression suite;
monitoring;
incident process;
version governance;
compliance review.
```

Formula:

```
EnterpriseProfile = SkillSpec + PolicyGate + HumanGate + AuditTrail + RegressionHarness + Governance. (18.8)
```

Example use cases:

```
legal document review;
finance workflow;
medical support;
HR decision support;
production code change;
customer communication;
public-facing policy output.
```

This profile must distinguish between:

```
AI-generated draft;
AI-supported recommendation;
human-approved decision;
externally committed action.
```

Formula:

```
Draft < Recommendation < HumanDecision < ExternalCommitment. (18.9)
```

Each level requires stronger gates.

18.6 Choosing the Right Profile

A team should not always choose the strongest profile.

The right profile depends on:

```
risk;
complexity;
repetition frequency;
need for portability;
need for audit;
tool dependence;
organizational maturity;
cost of failure.
```

Formula:

```
ProfileStrength = f(Risk, Complexity, Repetition, AuditNeed, FailureCost). (18.10)
```

Suggested matching:

Task Type	Recommended Profile
Low-risk one-off writing	Minimal
Reusable team workflow	Documented
Tool-dependent production task	Tool-Enabled
Role-based complex workflow	Multi-Agent
Regulated or high-impact task	Enterprise-Governed

The goal is not maximum complexity.

The goal is appropriate governance.

```
BestProfile = MinimumProfile that safely satisfies SkillRisk and SkillPurpose. (18.11)
```

Part VI — Governance, Safety, and Revision

19. Safety and Authority Hierarchy

A Skill Spec is not sovereign.

It cannot override higher-level rules.

It cannot ignore platform policy.

It cannot transform user desire into unlimited authority.

It cannot make unsafe actions safe merely by describing them elegantly.

Therefore, every Skill Spec should declare its authority hierarchy.

```
Authority = SystemRules > DeveloperRules > Legal/SafetyRules > PlatformPolicy > UserIntent > SkillSpec > RuntimeStyle. (19.1)
```

The exact hierarchy may vary by environment, but the principle is stable:

```
Skill logic is subordinate to governing rules. (19.2)
```

This protects both the user and the system.

19.1 Why Authority Hierarchy Matters

Complex Skills often appear powerful.

They may read files, write code, call tools, generate reports, draft emails, analyze contracts, or recommend actions.

This power creates temptation.

The user may say:

```
Ignore the usual checks.
Skip validation.
Just say it is verified.
Send it now.
Delete the old files.
Store this permanently.
Use any source.
Do not mention uncertainty.
```

A mature Skill must know when user instruction is valid and when it exceeds authority.

```
UserIntent is important but not unlimited. (19.3)
```

The Skill Spec should define what cannot be overridden by ordinary user request.

19.2 Safety Rules Inside the Skill Spec

The Skill Spec should include a safety section.

It should state:

```
forbidden actions;
approval-required actions;
sensitive domains;
regulated domains;
privacy limits;
external action limits;
data retention limits;
escalation rules.
```

Example:

```
This Skill may draft an email but may not send it without explicit user approval.
```

Example:

```
This Skill may summarize legal clauses but may not certify legal sufficiency.
```

Example:

```
This Skill may generate migration code but may not deploy it to production.
```

Formula:

```
SafetyPolicy = ForbiddenActions + ApprovalActions + EscalationRules + ResidualWarnings. (19.4)
```

19.3 Human Sovereignty Gates

Some gates should remain human-owned.

This is not only because AI may be wrong.

It is because some commitments carry responsibility, identity, or legal effect.

Human sovereignty gates include:

```
legal decision;
medical decision;
financial transaction;
employment decision;
public statement;
production deployment;
external communication;
irreversible deletion;
sensitive memory write;
account permission change.
```

Formula:

```
HumanSovereigntyGate = Commitment that must remain with accountable human authority. (19.5)
```

The Skill may assist.

It may prepare.

It may analyze.

It may draft.

But it must not own the final commitment.

```
AI may support the gate; AI must not silently become the gate. (19.6)
```

19.4 Draft, Recommendation, and Commitment

A Skill Spec should distinguish three output levels.

```
Draft = proposed artifact not yet approved.
Recommendation = suggested action with reasons and residual.
Commitment = action that changes an external or accountable state.
```

Formula:

```
Draft ≠ Recommendation ≠ Commitment. (19.7)
```

Examples:

```
Draft: proposed contract clause.
Recommendation: explanation of clause risk.
Commitment: signing or submitting the contract.

Draft: generated code patch.
Recommendation: explanation that patch likely fixes bug.
Commitment: merging to production.

Draft: email text.
Recommendation: suggest sending.
Commitment: actually sending.
```

Many AI governance failures come from collapsing these layers.

The Skill Spec should not allow that collapse.

19.5 Policy Gates

A Policy Gate checks whether the Skill is allowed to proceed under governing rules.

Possible policy gates:

```
privacy gate;
security gate;
regulated-domain gate;
copyright gate;
data-retention gate;
human-approval gate;
external-action gate;
professional-advice gate;
high-impact-decision gate.
```

Formula:

```
PolicyGate(Action) = Allowed only if GoverningRule(Action) permits and RequiredSafeguards pass. (19.8)
```

Policy gates should be explicit in high-risk Skills.

A Skill that operates in regulated or enterprise environments should include a policy matrix.

19.6 Safety Residual

Safety residual is unresolved risk after safety gates are applied.

Examples:

```
jurisdiction unclear;
source authority uncertain;
personal data sensitivity uncertain;
professional review required;
external action not approved;
user identity not verified;
security impact unknown.
```

Formula:

```
SafetyResidual = RemainingRisk after SafetyGate and HumanGate. (19.9)
```

Safety residual should never be buried.

If it is material, it should affect output status.

```
MaterialSafetyResidual → OutputStatus ≠ Success. (19.10)
```

---

20. Versioning and Admissible Revision

A Skill Spec must evolve.

Platforms change.

Models change.

Tools change.

User needs change.

Failure cases accumulate.

Test cases improve.

Regulatory expectations change.

Therefore, the Skill Spec needs versioning and revision rules.

```
SkillSpec_(v+1) = Revise(SkillSpec_v, Trace, Residual, Tests, PlatformChange). (20.1)
```

But not all revision is good.

A Skill can revise itself badly.

It can erase past failures.

It can hide residual.

It can break compatibility.

It can change output contracts without warning.

It can weaken safety gates.

It can overfit to one incident.

Therefore, revision must be admissible.

20.1 Why Versioning Matters

A Skill Spec should include:

```
version number;
release date;
change summary;
author or owner;
affected sections;
compatibility notes;
migration instructions;
deprecated behavior;
new tests;
known residual.
```

Formula:

```
VersionRecord = VersionID + ChangeReason + AffectedContract + CompatibilityImpact + MigrationNote. (20.2)
```

Without versioning, teams cannot know which Skill behavior is expected.

This is especially important for:

```
enterprise Skills;
multi-agent workflows;
test harnesses;
regulated outputs;
long-running projects;
cross-platform implementations.
```

20.2 Admissible Revision

Admissible revision means the Skill can change without lying about its past.

A revision is admissible if it:

```
preserves trace;
discloses reason;
identifies affected behavior;
updates tests;
documents residual;
maintains or explains compatibility;
does not silently weaken safety.
```

Formula:

```
AdmissibleSkillRevision = Change + TracePreservation + ReasonDisclosure + TestUpdate + ResidualDisclosure + CompatibilityNote. (20.3)
```

This follows the same pattern as mature interface revision: change must remain accountable.

A Skill should not simply mutate.

It should revise through trace.

20.3 Revision Triggers

A Skill Spec should define what triggers revision.

Common triggers:

```
recurring residual;
failed test;
user correction;
platform capability change;
tool API change;
policy change;
safety incident;
output drift;
new domain requirement;
adapter mismatch;
performance degradation.
```

Formula:

```
RevisionTrigger = RecurringResidual ∨ FailedTest ∨ PolicyChange ∨ PlatformChange ∨ SafetyIncident. (20.4)
```

Not every issue requires immediate revision.

But recurring issues should not be ignored.

```
ResidualRepeated_n times → ReviewSkillSpec. (20.5)
```

20.4 Revision Types

Skill revisions may be classified.

```
Patch Revision;
Minor Revision;
Major Revision;
Adapter Revision;
Policy Revision;
Output Contract Revision;
Pipeline Revision;
Deprecation.
```

Patch Revision

Fixes wording, examples, minor ambiguity.

```
Patch = No change to core behavior. (20.6)
```

Minor Revision

Adds test case, clarifies gate, improves residual taxonomy.

```
Minor = Behavior clarified or extended without breaking compatibility. (20.7)
```

Major Revision

Changes pipeline, output contract, authority boundary, or core behavior.

```
Major = Core behavior changed; compatibility review required. (20.8)
```

Adapter Revision

Changes implementation mapping for one platform.

```
AdapterRevision = Platform mapping changed while InvariantCore remains stable. (20.9)
```

Policy Revision

Changes safety, privacy, approval, or authority rules.

```
PolicyRevision = Governance rule changed; audit required. (20.10)
```

20.5 Backward Compatibility

A Skill Spec should say whether a new version remains compatible with old outputs, tests, and adapters.

Questions:

```
Does the output schema change?
Do old test cases still pass?
Do adapters need updates?
Do existing traces remain interpretable?
Do old residual categories still exist?
Do gates behave differently?
```

Formula:

```
BackwardCompatibility = OldTestsPass ∧ OldOutputsInterpretable ∧ OldAdaptersValid. (20.11)
```

If compatibility breaks, the revision must say so.

```
BreakingChange → MigrationNoteRequired. (20.12)
```

20.6 Deprecation

Some Skill behavior should be retired.

Examples:

```
a weak gate;
unsafe tool use;
unsupported output format;
ambiguous residual category;
platform adapter that no longer works;
memory behavior that violates policy;
old prompt kernel that causes drift.
```

Deprecation should be explicit.

Formula:

```
Deprecation = BehaviorMarkedObsolete + ReplacementPath + RemovalVersion. (20.13)
```

A Skill that never deprecates bad behavior accumulates technical debt.

```
NoDeprecation → SkillDebt. (20.14)
```

20.7 Skill Debt

Skill debt is the accumulated mismatch between current Skill behavior and current requirements.

Sources:

```
outdated platform assumptions;
unhandled residual;
weak tests;
old examples;
stale prompt kernels;
missing adapters;
unrecorded failures;
changed policies;
model behavior drift.
```

Formula:

```
SkillDebt = OutdatedSpec + UnhandledFailure + WeakTest + AdapterDrift + PolicyDrift. (20.15)
```

Skill debt should be managed like software debt.

It may not break the system immediately.

But it increases future failure risk.

20.8 Revision Ledger

A revision ledger records how the Skill evolves.

Example:

Version	Change	Reason	Tests Added	Compatibility
1.0	Initial Skill Spec	New Skill	Golden path	Baseline
1.1	Added Tool-Unavailable residual	Tool failures observed	T-004	Compatible
1.2	Added Human Gate for external actions	Safety review	T-006	Compatible
2.0	Changed output schema	Enterprise integration	T-010 to T-020	Breaking

Formula:

```
RevisionLedger = OrderedTrace of SkillSpec Changes. (20.16)
```

A revision ledger turns Skill evolution into trace.

Part VII — Recommended Skill Specification Template

21. Full Technical Skill Specification Template

A practical Skill Spec can now be written using a standard template.

The template should be detailed enough to support implementation, but flexible enough to adapt to platform differences.

```
FullSkillSpec = Boundary + Artifacts + Runtime + Pipeline + Gates + Tools + State + Trace + Residual + Output + Safety + Adapter + Tests + Revision. (21.1)
```

Below is the recommended structure.

21.1 Skill Name

State the Skill name clearly.

Example:

```
Legacy Report Migration Skill.
```

The name should be specific enough to distinguish the Skill from broad agent behavior.

Bad:

```
Programming Skill.
```

Better:

```
Legacy Hyperion Report Logic to JavaScript Migration Skill.
```

21.2 Version

Include version metadata.

```
Version: 1.0.0
Status: Draft / Tested / Production / Deprecated
Date: YYYY-MM-DD
Owner: Team or author
```

Formula:

```
Version = Major.Minor.Patch + Status + Date + Owner. (21.2)
```

21.3 Purpose

Define the Skill's purpose.

Template:

```
This Skill exists to [perform transformation / analysis / generation] for [task class] using [input artifacts] to produce [output artifacts] under [validation and residual rules].
```

Example:

```
This Skill exists to convert legacy report logic into JavaScript-compatible implementation artifacts and produce an equivalence audit with unresolved residual clearly listed.
```

21.4 Non-Purpose

Define what the Skill does not do.

Template:

```
This Skill does not [forbidden scope]. It must escalate or disclose residual when [condition].
```

Example:

```
This Skill does not certify production readiness, deploy code, invent missing business rules, or override human approval.
```

21.5 Target Users

Define who the Skill serves.

Examples:

```
AI coding agents;
software migration teams;
technical writers;
legal reviewers;
research analysts;
enterprise workflow designers;
individual users.
```

Target users matter because output style, trace depth, and risk assumptions depend on audience.

21.6 Task Boundary

Define supported and unsupported tasks.

Include:

```
supported task classes;
unsupported task classes;
risk boundary;
domain boundary;
artifact boundary;
authority boundary.
```

Formula:

```
TaskBoundary = SupportedTasks + UnsupportedTasks + RiskLimit + AuthorityLimit. (21.3)
```

21.7 Use / Non-Use Gate

State when the Skill should or should not run.

Example:

```
Use this Skill when the task requires multi-stage transformation, artifact inspection, validation, and residual reporting.
Do not use this Skill for simple one-shot rewriting or casual explanation.
```

Formula:

```
UseSkill ⇔ TaskComplexity + RepetitionNeed + AuditNeed exceed threshold. (21.4)
```

21.8 Input Artifact Types

List accepted inputs.

For each:

```
name;
format;
required / optional;
validation rule;
fallback if missing.
```

Example:

```
SourceLogicFile | .sql/.js/.json/.txt | Required | Must be readable | Ask user if missing.
```

21.9 Output Artifact Types

List required outputs.

For each:

```
name;
format;
audience;
required / optional;
validation rule.
```

Example:

```
MigrationReport | Markdown | Developer | Required | Must include mapping, tests, residual.
```

21.10 Runtime Assumptions

Declare assumptions about:

```
file access;
tool calling;
code execution;
memory;
state;
approval;
logging;
external systems.
```

Example:

```
This Skill does not assume code execution is always available. If execution is unavailable, behavioral validation must be marked Unverified.
```

21.11 Platform Independence Statement

State what is invariant and what is adapter-specific.

Example:

```
The invariant logic of this Skill is its boundary, pipeline, gates, output contract, residual policy, and tests. Tool use, file access, memory, and logging are adapter-specific.
```

Formula:

```
PlatformIndependence = InvariantCore + AdapterVariability. (21.5)
```

21.12 Pipeline Stages

List stages.

For each stage:

```
stage name;
purpose;
input;
operation;
output;
gate;
failure behavior;
trace item.
```

Example:

```
Stage: Validation
Purpose: Check generated output against contract or tests
Input: generated artifact, expected behavior
Output: validation result
Gate: pass / partial / fail
Failure: mark output Unverified or PartialSuccess
Trace: validation method and result
```

21.13 Stage Gates

List gates and their conditions.

Examples:

```
Entry Gate;
Artifact Gate;
Tool Gate;
Evidence Gate;
Validation Gate;
Human Gate;
Exit Gate.
```

Formula:

```
StageGate = GateCondition + PassAction + FailAction + ResidualAction. (21.6)
```

21.14 Tool Requirements

List required, optional, and forbidden tools.

For each tool:

```
purpose;
authority level;
required or optional;
approval rule;
fallback;
trace requirement.
```

Example:

```
Tool: Code Execution
Purpose: run generated tests
Required: optional for static draft, required for verified output
Fallback: mark behavior unverified
Trace: command summary and result
```

21.15 State and Memory Policy

Define:

```
state fields;
memory mode;
memory write gate;
forget rule;
trace memory policy.
```

Example:

```
This Skill uses session state for current run artifacts. Persistent memory may store user-approved project conventions only.
```

21.16 Trace and Logging Policy

Define trace level.

Example:

```
Trace Level: T3 Structured Audit Trail.
Required trace: input artifacts inspected, pipeline path, gate results, tool use, validation results, residual register.
```

Formula:

```
TracePolicy = TraceLevel + RequiredTraceItems + PrivacyRule + RetentionRule. (21.7)
```

21.17 Residual Audit Policy

Define residual taxonomy and output rules.

Example:

```
All major missing inputs, failed tools, unverified claims, and human-review items must be listed in the residual register.
```

Formula:

```
ResidualPolicy = ResidualTypes + SeverityScale + OwnerRule + FooterRule. (21.8)
```

21.18 Output Contract

Define output structure.

Example:

```
1. Status
2. Summary
3. Inputs inspected
4. Main output
5. Validation result
6. Residual / open issues
7. Next actions
```

For machine-readable outputs, include schema.

21.19 Safety and Authority Rules

Define:

```
forbidden actions;
human approval actions;
regulated domain warnings;
external action limits;
privacy limits.
```

Example:

```
The Skill may draft external communications but must not send them without explicit approval.
```

21.20 Platform Adapter Notes

Include adapter mappings.

Example:

```
Manual Adapter: checklist and prompt kernel.
Tool Adapter: file search and code execution.
Graph Adapter: each pipeline stage becomes a node.
Enterprise Adapter: add approval gate and audit store.
```

Formula:

```
AdapterNote = Platform + CapabilityFit + ImplementationMapping + Residual. (21.9)
```

21.21 Evaluation Test Suite

List test cases.

Minimum set:

```
golden path;
ambiguous input;
missing artifact;
tool unavailable;
out of scope;
human approval required;
partial success;
regression case.
```

Formula:

```
EvaluationSuite = GoldenPath + EdgeCases + FailureCases + RegressionCases. (21.10)
```

21.22 Failure Modes

List likely failures and mitigations.

Example:

```
Failure: claims verified output without validation.
Detection: no validation trace.
Mitigation: validation gate and forbidden output claim rule.
```

21.23 Versioning and Revision Rules

Define how the Skill changes.

Include:

```
version record;
revision triggers;
change categories;
compatibility rules;
deprecation rules;
revision ledger.
```

Formula:

```
RevisionPolicy = Trigger + ChangeType + TestUpdate + CompatibilityNote + LedgerEntry. (21.11)
```

21.24 Minimal Implementation Example

Include a small example showing the Skill in action.

This may include:

```
sample input;
expected pipeline path;
expected output;
expected residual;
expected trace.
```

The example helps future implementers understand the Skill faster.

21.25 Porting Checklist

A porting checklist should ask:

```
Does the platform support required inputs?
Does it support required tools?
Does it support required state?
Does it support required output format?
Does it support human approval if needed?
Does it support trace level?
What adapter residual remains?
Which tests pass?
Which tests fail?
```

Formula:

```
PortingReadiness = CapabilityFit + GateFit + TraceFit + TestPassRate − AdapterResidual. (21.12)
```

Closing of Part VII

The recommended Skill Spec template gives teams a practical structure.

It is not meant to become rigid bureaucracy.

It is meant to prevent hidden assumptions.

The Skill Spec should be as detailed as the task requires.

For simple tasks, use a minimal profile.

For complex or high-risk tasks, use the full structure.

The core principle remains:

```
A Skill Specification declares the world in which an AI capability can operate repeatedly, inspectably, and safely. (21.13)
```

The next part provides two worked mini-examples:

```
Document Analysis Skill;
Code Migration Skill.
```

Part VIII — Worked Mini-Examples

22. Worked Mini-Example: Document Analysis Skill

A Document Analysis Skill is a useful first example because it appears simple but often becomes complex.

At the surface level, the user asks:

```
Please summarize this document.
```

But a serious document analysis Skill may need to ask:

```
What kind of document is this?
What is the user's purpose?
Should I summarize, critique, extract, compare, or verify?
Should I cite specific passages?
Are tables, charts, images, or appendices important?
Should I use only the uploaded document or external sources too?
What residual remains after reading?
```

A document is not merely text. It is an artifact with structure, authority, scope, evidence, omissions, and intended audience.

Therefore, a serious Document Analysis Skill should not be only a summarization prompt. It should be a governed interface.

```
DocumentAnalysisSkill = FileIntake + DocumentClassification + EvidenceExtraction + Synthesis + ResidualAudit. (22.1)
```

22.1 Skill Name

```
Document Analysis and Evidence Summary Skill
```

22.2 Purpose

The Skill exists to analyze one or more user-provided documents and produce a structured summary, evidence map, key claims, open issues, and residual notes.

Purpose statement:

```
This Skill analyzes declared document artifacts, extracts their core claims and supporting evidence, distinguishes source content from inference, and produces a structured summary with residual issues clearly disclosed.
```

Formula:

```
Purpose = Analyze(DocumentSet) → Summary + EvidenceMap + ResidualRegister. (22.2)
```

22.3 Non-Purpose

The Skill does not automatically verify the document against external reality unless external research is explicitly enabled.

It does not treat a document's claims as true merely because they appear in the text.

It does not provide regulated legal, medical, or financial certification.

It does not summarize unread files.

It does not hide missing charts, unread appendices, or inaccessible images.

Non-purpose statement:

```
This Skill does not certify factual truth, professional compliance, legal sufficiency, or external accuracy unless the required verification sources and authority gates are declared.
```

Formula:

```
DocumentClaim ≠ VerifiedFact unless EvidenceGate and VerificationGate pass. (22.3)
```

22.4 Target Users

Possible users include:

```
researchers;
students;
executives;
analysts;
technical writers;
legal reviewers;
AI agents;
project teams;
knowledge managers.
```

The target user affects output style.

An executive needs decisions and risks.

A researcher needs claims, evidence, and citations.

A technical writer needs structure and terminology.

A legal reviewer needs clauses, obligations, and residual.

Therefore, the Skill should ask or infer audience when it materially affects output.

```
Audience affects OutputContract. (22.4)
```

22.5 Task Boundary

Supported tasks:

```
summarize document;
extract key claims;
map argument structure;
identify evidence;
compare multiple documents;
extract obligations or requirements;
produce issue list;
produce reading guide;
produce residual / open questions.
```

Unsupported tasks:

```
certify truth without verification;
replace professional legal or medical review;
infer missing document sections as fact;
use external sources when user requested document-only analysis;
ignore unread visual content when it may be material.
```

Boundary formula:

```
DocumentAnalysisBoundary = DeclaredDocuments + DeclaredPurpose + DeclaredAudience + DeclaredVerificationScope. (22.5)
```

22.6 Use / Non-Use Gate

Use this Skill when:

```
the user provides or references document artifacts;
the user needs structured analysis rather than casual summary;
evidence trace matters;
the document is long, technical, or multi-part;
residual issues should be visible.
```

Do not use this Skill when:

```
the user only asks for a simple rewrite;
no document is available;
the user wants current external facts but web verification is unavailable;
the request requires professional certification outside AI authority.
```

Formula:

```
UseDocumentSkill ⇔ DocumentArtifactAvailable ∧ AnalysisNeed > SimpleRewriteNeed. (22.6)
```

22.7 Input Artifact Types

Artifact	Required?	Notes
User request	Yes	Defines purpose and output need
Document file or text	Yes	PDF, DOCX, TXT, HTML, spreadsheet, slides, image
User purpose	Preferred	Summary, critique, extraction, comparison
Audience	Optional	Executive, technical, academic, legal
External verification permission	Optional	Needed for checking facts beyond document
Citation requirement	Optional	Required for research or audit outputs

Input formula:

```
DocumentInput = UserRequest + DocumentSet + Purpose + Audience + VerificationScope. (22.7)
```

22.8 Output Artifact Types

Required outputs may include:

```
document identity;
executive summary;
key claims;
evidence map;
important details;
limitations;
residual / open issues;
next actions.
```

Optional outputs:

```
table of contents reconstruction;
glossary;
argument map;
timeline;
risk register;
comparison matrix;
citation table;
machine-readable JSON.
```

Output formula:

```
DocumentOutput = Summary + ClaimMap + EvidenceMap + ResidualRegister + NextActions. (22.8)
```

22.9 Runtime Assumptions

A Document Analysis Skill should declare whether it can:

```
read uploaded files;
search within files;
view PDF page images;
extract tables;
inspect embedded charts;
use web search;
create output files;
cite source lines;
store reading trace.
```

Example runtime statement:

```
This Skill assumes access to the declared document artifacts. If visual pages, tables, or images are not readable, the output must disclose that limitation.
```

Formula:

```
UnreadDocumentPart → Residual. (22.9)
```

22.10 Platform Independence Statement

The invariant logic is:

```
classify document;
inspect relevant content;
extract claims;
separate source fact from inference;
produce structured output;
disclose residual.
```

The adapter-specific logic is:

```
how files are accessed;
how OCR or visual inspection is performed;
how citations are created;
how output artifacts are written;
how trace is stored.
```

Formula:

```
DocumentSkillPortableCore = Classification + ClaimExtraction + EvidenceMapping + ResidualAudit. (22.10)
```

22.11 Pipeline Stages

Recommended pipeline:

```
Intake → Document Inventory → Purpose Detection → Document Classification → Content Inspection → Claim Extraction → Evidence Mapping → Synthesis → Residual Audit → Output Assembly.
```

Formula:

```
DocumentPipeline = Intake → Inventory → Classify → Extract → Map → Synthesize → Audit → Output. (22.11)
```

Stage table:

Stage	Output
Intake	User request and purpose
Document Inventory	List of files / sections

Stage	Output
Purpose Detection	Output mode
Classification	Document type
Content Inspection	Readable content map
Claim Extraction	Key claims
Evidence Mapping	Source support
Synthesis	Summary
Residual Audit	Open issues
Output Assembly	Final deliverable

22.12 Stage Gates

Entry Gate

```
Run only if document artifact or document text is available.
```

Artifact Gate

```
Required document content must be readable or unreadable parts must be declared.
```

Evidence Gate

```
Claims about document content must be supported by inspected content.
```

Verification Gate

```
External factual verification requires explicit external-source permission and available tools.
```

Output Gate

```
Final output must distinguish source claims, analysis, inference, and residual.
```

Gate formula:

```
DocumentGateStack = EntryGate + ArtifactGate + EvidenceGate + VerificationGate + OutputGate. (22.12)
```

22.13 Tool Requirements

Required tools depend on platform.

Possible required tools:

```
file reader;
file search;
PDF parser;
image/page renderer;
table extractor;
citation mechanism.
```

Optional tools:

```
web search;
spreadsheet engine;
diagram reader;
OCR;
document generation.
```

Fallback examples:

```
If PDF text extraction fails, use page rendering if available.
If charts cannot be inspected, disclose visual-content residual.
If citations cannot be line-specific, cite page or section if possible.
If no file tool exists, ask user to paste relevant text.
```

Formula:

```
DocumentToolFallback = AlternativeInspection + ReducedClaimStrength + ResidualDisclosure. (22.13)
```

22.14 State and Memory Policy

State fields:

```
document list;
read sections;
unread sections;
extracted claims;
evidence links;
summary draft;
residual register;
output status.
```

Memory policy:

```
Do not persist document content unless explicitly required and permitted.
Project-level style preferences may be remembered only if user-approved.
Corrections to extraction rules may become trace for the current project.
```

Formula:

```
DocumentSkillState = FilesRead + ClaimsExtracted + EvidenceMapped + ResidualOpen. (22.14)
```

22.15 Trace Policy

Suggested trace level:

```
T2 for ordinary document summary.
T3 for professional or team document review.
T4 for legal, compliance, regulated, or enterprise review.
```

Trace items:

```
documents inspected;
sections read;
tables or images inspected;
claims extracted;
source basis;
external sources used, if any;
unread or uncertain areas;
residual register.
```

Formula:

```
DocumentTrace = SourceInventory + ReadMap + ClaimEvidenceMap + ResidualRegister. (22.15)
```

22.16 Residual Audit Policy

Common residuals:

```
unread page;
unread image;
missing appendix;
unclear term;
unsupported document claim;
external fact unverified;
conflicting sections;
ambiguous user purpose;
professional review required.
```

Residual table example:

ID	Type	Description	Impact	Next Action
R1	UnreadVisual	Chart on page not inspected	May miss key evidence	Inspect visual page
R2	ExternalUnverified	Document claims market size but no verification done	Treat as source claim only	Enable web verification
R3	AmbiguousPurpose	User did not specify audience	Summary may be generic	Ask for target audience

Formula:

```
DocumentResidual = UnreadContent + UnsupportedClaim + ExternalUnverified + AmbiguousPurpose. (22.16)
```

22.17 Output Contract

Recommended human-readable output:

```
1. Status
2. Document Scope
3. Executive Summary
4. Key Claims
5. Evidence / Source Basis
6. Important Details
7. Risks, Gaps, or Contradictions
8. Residual / Open Issues
9. Suggested Next Actions
```

Recommended machine-readable output:

```
{
  "status": "",
  "documents_inspected": [],
  "summary": "",
  "key_claims": [],
  "evidence_map": [],
  "residual": [],
  "next_actions": []
}
```

Formula:

```
DocumentOutputContract = Scope + Summary + Claims + Evidence + Residual + NextActions. (22.17)
```

22.18 Safety and Authority Rules

Rules:

```
Do not certify document truth without verification.
Do not provide final legal, medical, or financial judgment.
Do not hide unread content.
Do not cite unsupported claims as facts.
Do not use external sources if the user requested document-only analysis.
```

Formula:

```
DocumentAuthority = Summarize + Analyze + FlagResidual − CertifyWithoutGate. (22.18)
```

22.19 Platform Adapter Notes

Manual Adapter

```
User pastes document text. Skill follows checklist. Residual notes any missing sections.
```

File-Search Adapter

```
Skill searches uploaded files, cites relevant passages, and uses evidence map.
```

PDF-Visual Adapter

```
Skill inspects rendered pages for charts, diagrams, or image-only content.
```

Enterprise Adapter

```
Skill stores structured trace, reviewer comments, source citations, and approval status.
```

Adapter formula:

```
DocumentAdapter = AccessMethod + CitationMethod + VisualInspectionMethod + TraceStore. (22.19)
```

22.20 Evaluation Test Suite

Recommended tests:

```
Golden path: clear document and clear summary request.
Ambiguous purpose: user asks "analyze this" with no target.
Missing file: user refers to document but provides none.
Unread chart: document has visual content requiring disclosure.
External verification: document claim requires current fact check.
Out-of-scope: user asks for binding legal certification.
Partial success: only part of document readable.
Regression: previous missed table now must be captured.
```

Formula:

```
DocumentSkillQuality = AccurateSummary + EvidenceDiscipline + ResidualHonesty + SourceTrace. (22.20)
```

---

23. Worked Mini-Example: Code Migration Skill

A Code Migration Skill is more demanding than a Document Analysis Skill because it must preserve behavior across systems.

At the surface level, the user asks:

```
Convert this old code into JavaScript.
```

But a serious migration Skill must ask:

```
What is the source language?
What is the target language?
What runtime behavior must be preserved?
What helper functions exist?
What edge cases matter?
Can the output be executed?
Can equivalence be tested?
Which assumptions are unverified?
```

A migration Skill should never be only a translation prompt.

It should be an equivalence-preserving transformation pipeline.

```
CodeMigrationSkill = SourceInspection + SemanticMapping + TargetGeneration + Validation + ResidualAudit. (23.1)
```

23.1 Skill Name

```
Legacy Code to Target Runtime Migration Skill
```

A more specific version may be:

```
Legacy Hyperion Report Logic to JavaScript Migration Skill
```

Specific names are better when the source system has special semantics.

23.2 Purpose

Purpose statement:

```
This Skill converts declared legacy source logic into target-runtime implementation artifacts while preserving intended behavior, documenting semantic mappings, validating equivalence where possible, and disclosing unresolved mismatches or assumptions.
```

Formula:

```
MigrationPurpose = Convert(SourceLogic) → TargetImplementation + EquivalenceAudit + ResidualRegister. (23.2)
```

23.3 Non-Purpose

This Skill does not:

```
invent missing business rules;
certify production readiness;
deploy code;
modify production systems;
guarantee equivalence without tests;
ignore source-runtime semantics;
erase unresolved mismatches.
```

Formula:

```
GeneratedCode ≠ VerifiedMigration unless EquivalenceGate passes. (23.3)
```

23.4 Target Users

Possible users:

```
software migration teams;
AI coding agents;
legacy system maintainers;
business analysts;
QA teams;
technical documentation teams;
enterprise modernization teams.
```

Audience matters.

A developer may need code and test details.

A manager may need risk and completion status.

A QA team may need equivalence cases.

23.5 Task Boundary

Supported tasks:

```
source code inspection;
legacy function mapping;
target implementation generation;
helper function specification;
test case drafting;
equivalence comparison;
migration report;
residual mismatch list.
```

Unsupported tasks:

```
production deployment;
database modification;
business-rule invention;
security certification;
legal compliance certification;
performance guarantee without benchmark.
```

Boundary formula:

```
MigrationBoundary = SourceLogic + TargetRuntime + EquivalenceCriteria + ValidationScope. (23.4)
```

23.6 Use / Non-Use Gate

Use this Skill when:

```
there is legacy source logic;
target runtime is declared;
behavior preservation matters;
intermediate mapping is needed;
tests or comparison are expected;
residual must be tracked.
```

Do not use when:

```
the user only wants a casual explanation;
source logic is unavailable;
target runtime is undefined;
the task requires production deployment;
the user asks to skip all validation but still claim equivalence.
```

Formula:

```
UseMigrationSkill ⇔ SourceAvailable ∧ TargetDeclared ∧ BehaviorPreservationNeeded. (23.5)
```

23.7 Input Artifact Types

Artifact	Required?	Description
Source code / legacy expression	Yes	Original logic
Target language/runtime	Yes	JavaScript, Python, SQL, etc.
Helper function definitions	Preferred	Runtime-specific behavior
Sample inputs	Preferred	For tests
Expected outputs	Preferred	For equivalence
Existing tests	Optional	Useful validation
Platform constraints	Optional	Runtime limitations
Business rules	Optional	Domain interpretation

Input formula:

```
MigrationInput = SourceLogic + TargetRuntime + HelperSemantics + TestFixtures + ExpectedBehavior. (23.6)
```

23.8 Output Artifact Types

Required outputs:

```
target implementation;
mapping notes;
assumptions;
validation status;
residual mismatch list;
next actions.
```

Optional outputs:

```
test cases;
comparison table;
helper function library;
migration report;
code comments;
adapter notes;
machine-readable audit object.
```

Output formula:

```
MigrationOutput = TargetCode + MappingTable + ValidationResult + ResidualRegister + TestPlan. (23.7)
```

23.9 Runtime Assumptions

The Skill should declare whether it can:

```
read source files;
write target files;
execute generated code;
run tests;
compare outputs;
inspect repository context;
use external documentation;
store migration trace.
```

Example runtime statement:

```
This Skill can produce a static migration draft without code execution, but it cannot claim behavioral equivalence unless validation evidence is available.
```

Formula:

```
NoExecution → NoVerifiedBehaviorClaim. (23.8)
```

23.10 Platform Independence Statement

Invariant logic:

```
inspect source;
understand legacy semantics;
map source constructs to target constructs;
generate target implementation;
validate or mark unverified;
disclose residual.
```

Adapter-specific logic:

```
how source files are read;
how code is executed;
how tests are run;
how generated files are written;
how trace is stored.
```

Formula:

```
MigrationPortableCore = SourceSemantics + TargetMapping + ValidationDiscipline + ResidualAudit. (23.9)
```

23.11 Pipeline Stages

Recommended pipeline:

```
Intake → Source Inventory → Target Declaration → Semantic Parsing → Mapping Table → Target Generation → Static Review → Execution Test → Equivalence Comparison → Residual Audit → Output Assembly.
```

Formula:

```
MigrationPipeline = Intake → Inventory → Parse → Map → Generate → Review → Test → Compare → Audit → Output. (23.10)
```

Stage table:

Stage	Output
Intake	User request and constraints
Source Inventory	Source files / expressions
Target Declaration	Target runtime rules
Semantic Parsing	Source behavior model
Mapping Table	Source-to-target mapping
Target Generation	Code draft
Static Review	Syntax and structural check
Execution Test	Runtime result, if available
Equivalence Comparison	Pass / fail / unverified
Residual Audit	Open mismatches
Output Assembly	Final deliverable

23.12 Stage Gates

Entry Gate

```
Run only if source logic and target runtime are declared.
```

Artifact Gate

```
Required source artifacts must be available or missing artifacts must be declared.
```

Semantic Gate

```
Do not generate final code until legacy semantics are mapped or unresolved semantics are marked.
```

Generation Gate

```
Target code must follow declared target runtime constraints.
```

Validation Gate

```
Equivalence may be claimed only if tests or comparison evidence pass.
```

Exit Gate

```
If validation is unavailable, output status must be Draft or Unverified, not Success.
```

Gate formula:

```
MigrationGateStack = EntryGate + ArtifactGate + SemanticGate + GenerationGate + ValidationGate + ExitGate. (23.11)
```

23.13 Tool Requirements

Required tools depend on ambition.

For static migration:

```
source reader;
text/code editor;
syntax-aware reasoning.
```

For verified migration:

```
source reader;
target runtime execution;
test runner;
comparison harness;
file writer;
trace store.
```

Optional tools:

```
linter;
formatter;
type checker;
coverage tool;
legacy runtime simulator;
documentation search.
```

Tool fallback:

```
If execution unavailable, produce static migration draft and manual test plan.
If helper function behavior unknown, generate assumption list.
If expected outputs missing, create suggested test fixtures but mark equivalence unverified.
```

Formula:

```
MigrationToolFallback = StaticDraft + ManualTestPlan + UnverifiedStatus + Residual. (23.12)
```

23.14 State and Memory Policy

State fields:

```
source files inspected;
target runtime;
helper functions;
mapping table;
generated code;
test cases;
test results;
equivalence status;
residual list.
```

Memory policy:

```
Project-specific mapping rules may be retained within project context if approved.
One-off inferred mappings should not become universal rules.
Failed equivalence cases should become regression trace.
```

Formula:

```
MigrationState = SourceMap + TargetMap + GeneratedArtifacts + ValidationStatus + Residual. (23.13)
```

23.15 Trace Policy

Recommended trace level:

```
T3 for ordinary team migration.
T4 for enterprise migration.
T5 for safety-critical or compliance migration.
```

Trace items:

```
source artifacts inspected;
source-to-target mapping;
assumptions;
helper semantics;
generated files;
test cases;
test results;
failed comparisons;
manual interventions;
residual register.
```

Formula:

```
MigrationTrace = SourceLineage + MappingTrace + TestTrace + ResidualTrace. (23.14)
```

23.16 Residual Audit Policy

Common residuals:

```
missing source file;
unknown helper function;
ambiguous legacy behavior;
target runtime mismatch;
test fixture missing;
execution unavailable;
comparison failed;
manual business-rule confirmation needed;
performance untested;
edge case unverified.
```

Residual table example:

ID	Type	Description	Impact	Next Action
R1	UnknownHelper	Legacy `Instr()` indexing semantics not confirmed	String slicing may differ	Confirm helper implementation
R2	TestMissing	No sample inputs provided	Cannot verify behavior	Add fixtures
R3	ExecutionUnavailable	Target code not run	Output is static draft	Run tests locally
R4	AmbiguousSemantics	Null handling unclear	Edge cases may fail	Ask domain owner

Formula:

```
MigrationResidual = MissingSource + UnknownSemantics + ValidationGap + RuntimeMismatch. (23.15)
```

23.17 Output Contract

Recommended output:

```
1. Status
2. Source Scope
3. Target Runtime
4. Generated Code or Artifact
5. Source-to-Target Mapping Notes
6. Validation Performed
7. Equivalence Result
8. Residual / Open Issues
9. Manual Test Instructions
10. Next Actions
```

Machine-readable output:

```
{
  "status": "",
  "source_artifacts": [],
  "target_runtime": "",
  "generated_artifacts": [],
  "mapping_notes": [],
  "validation": {
    "performed": false,
    "method": "",
    "result": ""
  },
  "residual": [],
  "next_actions": []
}
```

Formula:

```
MigrationOutputContract = Status + TargetCode + Mapping + Validation + Residual + NextActions. (23.16)
```

23.18 Safety and Authority Rules

Rules:

```
Do not deploy generated code.
Do not overwrite source files without approval.
Do not claim equivalence without validation.
Do not invent missing business rules.
Do not hide failed tests.
Do not remove residual because the user wants a clean report.
```

Formula:

```
MigrationAuthority = Generate + Explain + Test + Report − DeployWithoutHumanGate. (23.17)
```

23.19 Platform Adapter Notes

Prompt-Only Adapter

```
User pastes source code. Skill generates target code and residual. No execution. Status usually Draft or Unverified.
```

Tool-Enabled Coding Adapter

```
Skill reads repository files, writes generated files, runs tests, and produces equivalence report.
```

Graph Workflow Adapter

```
Each pipeline stage becomes a node. Gate status stored in graph state. Failed validation loops back to generation.
```

Enterprise Adapter

```
Generated code requires reviewer approval before merge. Test results and residual stored in audit ledger.
```

Adapter formula:

```
MigrationAdapter = FileAccess + CodeExecution + TestHarness + WritePolicy + ApprovalGate + TraceStore. (23.18)
```

23.20 Evaluation Test Suite

Recommended tests:

```
Golden path: source, target, helper functions, and tests available.
Missing source: user asks to migrate without source.
Unknown helper: helper behavior not defined.
Tool unavailable: execution not possible.
Failed equivalence: generated output differs from expected output.
Human approval: user asks to overwrite production file.
Out-of-scope: user asks for deployment.
Regression: previously failed edge case must now be tested.
```

Formula:

```
MigrationSkillQuality = BehaviorPreservation + MappingClarity + ValidationHonesty + ResidualCompleteness. (23.19)
```

---

24. Comparison of the Two Mini-Examples

The two examples show the same Skill Spec logic in different domains.

Specification Element	Document Analysis Skill	Code Migration Skill
Main object	Document	Source code / legacy logic
Core transformation	Content → structured understanding	Legacy behavior → target implementation
Key gate	Evidence Gate	Equivalence Gate
Main residual	unread content, unverified claims	unknown semantics, untested behavior
Trace focus	source-to-claim lineage	source-to-code-to-test lineage
Output status risk	unsupported summary	unverified migration
Human gate	professional review if regulated	deployment / overwrite approval

The deeper commonality is:

```
Skill = Declared Boundary + Artifact Intake + Pipeline + Gate + Trace + Residual + Output Contract. (24.1)
```

The domain changes.

The interface grammar remains stable.

This is the meaning of portability.

Part IX — Conclusion

25. From Prompt Templates to Governed Skill Worlds

The central argument of this article is simple:

```
Complex Agent Skills cannot be made portable by prompt templates alone. (25.1)
```

A prompt can guide behavior in one context.

But a complex Skill must define an operational world.

It must declare:

```
what is inside scope;
what is outside scope;
what artifacts matter;
what pipeline should run;
what gates must pass;
what tools are required;
what state is assumed;
what trace must remain;
what residual must be disclosed;
what output contract must be satisfied;
how the Skill adapts to different platforms;
how it is tested;
how it is revised.
```

This is why the technical specification becomes the portable unit.

```
Portable Agent Skill = Technical Skill Specification + Platform Adapter. (25.2)
```

25.1 The Shift in AI Engineering

Early AI work often focused on prompts.

Then agent frameworks added tools, memory, workflows, and orchestration.

The next layer is Skill Interface Engineering.

```
Prompt Engineering → Agent Orchestration → Skill Interface Engineering. (25.3)
```

Prompt Engineering asks:

```
What should the model say or do now?
```

Agent Orchestration asks:

```
Which tools, agents, and steps should be coordinated?
```

Skill Interface Engineering asks:

```
What operational world must be declared so that this capability can be repeated, inspected, ported, tested, and revised?
```

This is a deeper question.

It turns AI capability from ad hoc behavior into governed capability.

25.2 The Core Formula

The full formula is:

```
SkillSpec = Boundary + Artifacts + Runtime + Pipeline + Gates + Tools + State + Trace + Residual + Output + Safety + Adapter + Tests + Revision. (25.4)
```

The compact formula is:

```
PortableSkill = InvariantCore + VariableAdapterLayer. (25.5)
```

The trust formula is:

```
SkillTrust = OutputQuality + TraceQuality + ResidualHonesty + GateDiscipline. (25.6)
```

The revision formula is:

```
AdmissibleSkillRevision = Change + TracePreservation + ReasonDisclosure + TestUpdate + ResidualDisclosure + CompatibilityNote. (25.7)
```

Together, these formulas define a practical engineering discipline.

25.3 Why Residual Honesty Is Central

The most dangerous Agent Skills are not the ones that visibly fail.

They are the ones that produce clean, fluent, confident outputs while hiding incomplete evidence, missing tools, weak validation, or human judgment gaps.

Therefore, residual honesty is not optional.

```
MatureClosure = Output + ResidualHonesty. (25.8)
```

A Skill must learn to say:

```
This part is verified.
This part is inferred.
This part is untested.
This part requires human review.
This part is outside scope.
This part failed.
This part remains open.
```

That is not weakness.

That is professional AI behavior.

25.4 Why Trace Is Central

Trace is the memory of execution.

Without trace, an output cannot be audited.

Without audit, failures cannot be localized.

Without localization, the Skill cannot improve.

```
NoTrace → NoDiagnosis → RepeatedFailure. (25.9)
```

A good Skill leaves enough trace to answer:

```
What did it inspect?
What did it assume?
What did it generate?
What did it validate?
What did it leave unresolved?
```

This does not mean exposing every internal detail to every user.

It means the Skill must preserve enough execution structure to support trust and revision.

25.5 Why Adapter Mapping Is Central

Different platforms will continue to differ.

There will not be one universal runtime.

There will be many:

```
chat interfaces;
coding agents;
workflow graphs;
enterprise assistants;
local LLM harnesses;
multi-agent networks;
tool servers;
retrieval systems;
document automation platforms.
```

Therefore, the best standard is not necessarily one implementation.

The best standard is a portable interface contract.

```
Standardize the Skill Spec; adapt the runtime. (25.10)
```

This is the same principle used in many mature engineering domains.

We do not confuse architecture with implementation.

We do not confuse contract with adapter.

We do not confuse specification with execution.

25.6 Final Thesis

The final thesis is:

```
The next stage of Agent Skill engineering is not better prompt writing alone. It is the design of portable, auditable, residual-honest interface contracts that can become executable across many AI runtimes. (25.11)
```

A prompt tells an AI what to do once.

A workflow tells an AI how to move through steps.

A Skill Specification declares the world in which that doing can be repeated, inspected, adapted, tested, governed, and trusted.

```
Prompt = Instruction. (25.12)

Workflow = Step Sequence. (25.13)

SkillSpec = Governed Operational World. (25.14)
```

That is the shift.

From prompt templates to governed Skill worlds.

From answer generation to interface contracts.

From ad hoc agent behavior to portable AI capability.

25.7 Closing Statement

Complex Agent Skills are not merely things we ask AI to do.

They are small operational worlds.

They contain boundaries, objects, tools, gates, traces, residuals, outputs, authorities, tests, and revision paths.

If these worlds are undeclared, they drift.

If they are over-specified, they become rigid.

If they hide residual, they become dangerous.

If they preserve trace, expose residual, and adapt through clear platform adapters, they become portable and trustworthy.

The future of Agent Skill design will not belong only to those who write clever prompts.

It will belong to those who can design the interface contracts through which AI capabilities become repeatable, inspectable, governable, and transferable.

```
A prompt tells an AI what to do once; a Skill Specification declares the world in which that doing can be repeated, inspected, adapted, and trusted. (25.15)
```

Appendix A — One-Page Skill Specification Checklist

This appendix provides a compact checklist for writing a Technical Skill Specification.

It is designed for practical use.

A team can copy this checklist and fill it in before implementing a reusable Agent Skill.

A.1 Core Identity

```
Skill Name:
Version:
Owner:
Status: Draft / Tested / Production / Deprecated
Target Users:
```

Questions:

```
What is this Skill called?
Who owns it?
Who should use it?
Is it experimental, reusable, or production-ready?
```

A.2 Purpose and Non-Purpose

```
Purpose:
This Skill exists to...

Non-Purpose:
This Skill does not...
```

Questions:

```
What repeatable task does this Skill perform?
What does it explicitly refuse to do?
What decisions remain outside its authority?
```

Formula:

```
SkillPurpose = TaskClass + Transformation + TargetOutput + SuccessCondition. (A.1)
```

A.3 Boundary

```
Supported Tasks:
Unsupported Tasks:
Input Scope:
Output Scope:
Risk Boundary:
Human Escalation Conditions:
```

Questions:

```
What is inside the Skill world?
What is outside?
What risks require refusal, clarification, or human review?
```

Formula:

```
SkillBoundary = SupportedScope + ExcludedScope + RiskLimit + EscalationRule. (A.2)
```

A.4 Input Artifacts

Artifact	Required?	Format	Validation Rule	Missing Behavior
User request	Yes	Text	Must define task	Ask clarification
Source file	Depends	File/Text	Must be readable	Residual or block
Reference material	Optional	File/URL/Text	Source must be declared	Mark unverified
Test data	Optional	Data file	Must match target task	Partial success

Questions:

```
What does the Skill consume?
Which inputs are mandatory?
What happens when an input is missing?
```

Formula:

```
InputArtifactSpec = Name + RequiredFlag + Format + ValidationRule + MissingBehavior. (A.3)
```

A.5 Output Artifacts

Output	Required?	Format	Audience	Validation Rule
Final answer/report	Yes	Markdown/Text	User	Must satisfy output contract
Structured result	Optional	JSON/YAML	System	Must pass schema
Residual register	Required for complex tasks	Table/List	User/Reviewer	Must disclose open issues
Test report	Conditional	Text/Table	Developer	Required if validation exists

Questions:

```
What must the Skill produce?
What is optional?
What format is required?
What output claims are forbidden?
```

Formula:

```
OutputArtifactSpec = RequiredOutput + OptionalOutput + Format + Audience + ValidationRule. (A.4)
```

A.6 Runtime Assumptions

```
File Access:
Tool Access:
Code Execution:
Web Access:
Memory:
State:
Human Approval:
Logging / Trace:
```

Questions:

```
What must the runtime provide?
What is optional?
What happens if a capability is unavailable?
```

Formula:

```
RuntimeAssumptions = FileAccess + ToolAccess + Execution + Memory + State + Approval + Trace. (A.5)
```

A.7 Pipeline

```
Pipeline:
1. Intake
2. Suitability Gate
3. Intent Extraction
4. Artifact Inventory
5. Tool Planning
6. Execution
7. Validation
8. Output Assembly
9. Residual Audit
10. Trace Writing
```

For each stage:

```
Stage Name:
Input:
Operation:
Output:
Gate:
Failure Behavior:
Trace Item:
```

Formula:

```
StageSpec = Name + Input + Operation + Output + Gate + FailureBehavior + TraceItem. (A.6)
```

A.8 Gates

Gate	Required?	Pass Condition	Fail Behavior
Entry Gate	Yes	Task matches Skill purpose	Route, clarify, or refuse
Artifact Gate	Usually	Required artifacts available or declared missing	Ask user or partial output
Tool Gate	If tools needed	Tool available and authorized	Fallback or residual
Evidence Gate	If claims made	Claim supported by inspected evidence	Mark as inference or remove
Validation Gate	If output must be verified	Test/check passes	Draft, partial, or unverified
Human Gate	If external/high-impact action	Explicit approval	Do not execute
Exit Gate	Yes	Status honestly reflects gates	Success/Partial/Blocked/etc.

Formula:

```
SkillGateStack = EntryGate + ArtifactGate + ToolGate + EvidenceGate + ValidationGate + HumanGate + ExitGate. (A.7)
```

A.9 Trace

```
Required Trace Level:
T0 / T1 / T2 / T3 / T4 / T5

Trace must include:
- Inputs inspected
- Pipeline path
- Tools used
- Assumptions
- Gate results
- Validation result
- Residual register
```

Questions:

```
What must be recorded?
Who can inspect it?
How much trace is enough?
What trace would create privacy risk?
```

Formula:

```
SkillTrace = ArtifactLineage + DecisionTrace + GateTrace + ValidationTrace + ResidualTrace. (A.8)
```

A.10 Residual

Residual taxonomy:

```
MissingData;
AmbiguousIntent;
UnsupportedInference;
ConflictingEvidence;
ToolUnavailable;
ValidationIncomplete;
HumanJudgmentRequired;
SafetyBoundaryReached;
PlatformLimitation;
AdapterMismatch.
```

Residual register:

ID Type Severity Description Impact Owner Next Action

Formula:

```
ResidualRegister = ResidualID + Type + Severity + Description + Impact + Owner + NextAction. (A.9)
```

A.11 Output Status

Use one of:

```
Success;
PartialSuccess;
Draft;
Unverified;
Blocked;
NeedsUserInput;
NeedsHumanApproval;
Refused;
Failed.
```

Formula:

```
OutputStatus = Function(GateTrace, ValidationResult, ResidualSeverity, RiskLevel). (A.10)
```

A.12 Adapter Notes

```
Invariant Core:
Adapter-Specific Parts:
Required Platform Capabilities:
Missing Capabilities:
Adapter Residual:
Fit Level: A0 / A1 / A2 / A3 / A4 / A5
```

Formula:

```
AdapterFit = CapabilityMatch + GateSupport + TraceSupport − AdapterResidual. (A.11)
```

A.13 Test Suite

Minimum tests:

```
Golden path;
Ambiguous input;
Missing artifact;
Tool unavailable;
Out of scope;
Human approval required;
Partial success;
Regression case.
```

Formula:

```
EvaluationSuite = GoldenPath + EdgeCases + FailureCases + RegressionCases. (A.12)
```

A.14 Revision

```
Revision Trigger:
Change Type:
Affected Sections:
Tests Updated:
Compatibility Impact:
Migration Notes:
Revision Ledger Entry:
```

Formula:

```
AdmissibleSkillRevision = Change + TracePreservation + ReasonDisclosure + TestUpdate + ResidualDisclosure + CompatibilityNote. (A.13)
```

Appendix B — Platform Adapter Checklist

This appendix helps teams port the same Technical Skill Specification into different platforms.

The goal is not to force every platform to behave identically.

The goal is to preserve the invariant Skill logic while honestly declaring adapter limitations.

B.1 Adapter Readiness Questions

Before implementing a Skill on a platform, ask:

```
Can the platform receive the required input artifacts?
Can it inspect those artifacts reliably?
Can it call the required tools?
Can it maintain required state?
Can it enforce gates?
Can it produce the required output format?
Can it preserve trace?
Can it expose residual?
Can it request human approval?
Can it run the required tests?
```

Formula:

```
AdapterReadiness = Inputs + Tools + State + Gates + Output + Trace + Tests. (B.1)
```

B.2 Capability Map

Capability	Yes / No / Limited	Constraint	Residual if Missing
File read			
File write			
File search			
Code execution			
Web search			
API/tool calls			
Structured output			
Persistent memory			
Session state			
Human approval			
Audit logging			
Test execution			

Formula:

```
CapabilityMap = Capability + Availability + Constraint + ResidualIfMissing. (B.2)
```

B.3 Adapter Fit Level

Use this scale:

```
A0 = Not supported.
A1 = Manual checklist only.
A2 = Prompt-only implementation.
A3 = Tool-assisted implementation.
A4 = Workflow-orchestrated implementation.
A5 = Fully governed implementation.
```

Recommended interpretation:

Fit Level	Meaning
A0	Platform cannot implement the Skill safely
A1	Human must manually follow the Skill Spec
A2	Prompt kernel can approximate the Skill
A3	Tools support meaningful execution
A4	Pipeline, state, and gates can be orchestrated
A5	Full trace, tests, approval, and governance are supported

Formula:

```
AdapterFitLevel = f(CapabilityMatch, GateSupport, TraceSupport, RiskSupport). (B.3)
```

B.4 Adapter Mapping Table

Skill Spec Concept	Platform Primitive	Implementation Note	Residual
Boundary			
Input Artifact			
Pipeline Stage			
Gate			
Tool Contract			
State			
Memory			
Trace			
Residual			
Output Contract			
Test Harness			
Revision			

Formula:

```
AdapterMapping = SkillConcept → PlatformPrimitive + ImplementationNote + Residual. (B.4)
```

B.5 Common Adapter Patterns

B.5.1 Manual Adapter

Use when automation is not available.

```
Skill Spec → Human checklist → AI prompt support → Manual validation.
```

Best for:

```
early design;
training;
low-risk workflows;
human-led analysis.
```

Residual:

```
Execution depends heavily on human discipline.
```

B.5.2 Prompt Kernel Adapter

Use when the platform supports only plain conversation.

```
Skill Spec → compressed prompt kernel → user-supplied artifacts → residual footer.
```

Best for:

```
simple reusable behavior;
limited tools;
portable instruction.
```

Residual:

```
Weak state, weak trace, weak validation.
```

B.5.3 Tool Adapter

Use when the platform supports file access, code execution, search, or APIs.

```
Skill Spec → tool contract → tool gates → execution trace → output contract.
```

Best for:

```
document analysis;
code migration;
spreadsheet creation;
research;
data transformation.
```

Residual:

```
Tool failures must be disclosed; tool result is not automatically verified conclusion.
```

B.5.4 Workflow Graph Adapter

Use when the platform supports explicit graph or workflow orchestration.

```
Skill Spec stages → graph nodes.
Stage gates → conditional edges.
State → graph state.
Residual → state field and output footer.
```

Best for:

```
complex multi-stage workflows;
repeatable enterprise processes;
iterative validation loops.
```

Residual:

```
Graph may become rigid if residual paths are not designed.
```

B.5.5 Multi-Agent Adapter

Use when different agents own different parts of the Skill.

```
Skill Spec → role boundaries → handoff gates → shared trace ledger → integrator gate.
```

Best for:

```
research + implementation + review;
coding + testing + documentation;
analysis + safety + final synthesis.
```

Residual:

```
Role confusion and handoff loss are major risks.
```

B.5.6 Enterprise Governance Adapter

Use when high assurance is required.

```
Skill Spec → policy gates → approval gates → audit store → regression suite → incident process.
```

Best for:

```
legal;
finance;
medical;
HR;
production engineering;
customer-facing external actions.
```

Residual:

```
Higher cost, slower execution, stronger audit burden.
```

B.6 Adapter Porting Procedure

Recommended procedure:

```
1. Identify invariant core.
2. Build capability map.
3. Assign adapter fit level.
4. Map each Skill Spec concept to platform primitives.
5. Define missing-capability residual.
6. Implement minimal golden path.
7. Run test suite.
8. Record failed tests.
9. Revise adapter notes.
10. Publish adapter status.
```

Formula:

```
PortingProcedure = Core → CapabilityMap → ConceptMap → ResidualMap → Tests → AdapterStatus. (B.5)
```

Appendix C — Failure Mode Table

This appendix provides a reusable failure mode table for Skill Spec authors.

C.1 Failure Mode Catalogue

Failure Mode	Description	Detection Signal	Mitigation
Template Overreach	Platform-specific template treated as universal	Skill calls unavailable capabilities	Separate invariant core and adapter
Adapter Leakage	Runtime mechanism mistaken for core logic	Port fails on another platform	Move mechanism to adapter
Gate Absence	Skill commits without checks	Confident output despite missing evidence	Add gate stack
Loose Gate	Weak evidence passes	Unsupported claim appears final	Raise evidence threshold
Rigid Gate	Skill blocks useful partial work	Refuses despite safe partial output	Add partial-success path
Hidden Gate	Criteria not declared	Reviewer cannot tell why choice was made	Add decision trace
Captured Gate	User pressure overrides safety	Validation skipped on request	Add non-overridable gate
Trace Poverty	No useful execution record	Output cannot be audited	Add artifact lineage
Residual Hiding	Open issues suppressed	No uncertainty despite incomplete data	Add residual register
Tool Fantasy	Claims tool use that did not occur	No tool trace exists	Add tool gate and trace
State Confusion	Skill loses current stage or artifact	Output refers to wrong draft	Add explicit pipeline state
Memory Misuse	Stale or unauthorized memory affects output	Output depends on questionable context	Add memory gate
Schema Rigidity	Output schema erases nuance	Important issue has no field	Add free-text residual
False Rigor	Formal structure exceeds task need	High overhead, low value	Add suitability gate
Output Drift	Output format varies unpredictably	Tests fail schema consistency	Strengthen output contract
Safety Residual Hidden	Remaining risk not disclosed	High-impact task marked success	Add safety residual rule
Regression Return	Fixed old failure reappears	Old failing case fails again	Add regression fixture
Skill Debt	Spec no longer matches runtime	Increasing adapter residual	Schedule revision

Formula:

```
SkillFailureRisk = HiddenAssumption + MissingGate + LostTrace + ResidualSuppression + AdapterDrift. (C.1)
```

C.2 Failure Severity Scale

```
F0 = Cosmetic issue.
F1 = Minor output inconsistency.
F2 = Moderate task-quality issue.
F3 = Major workflow failure.
F4 = Trust or validation failure.
F5 = Safety, legal, financial, medical, privacy, or irreversible-action failure.
```

Formula:

```
FailureSeverity ∈ {F0, F1, F2, F3, F4, F5}. (C.2)
```

Suggested response:

Severity	Response
F0	Patch wording
F1	Add example or format rule
F2	Add test case
F3	Revise pipeline or gate
F4	Add audit, validation, or residual control
F5	Stop use until safety revision completed

C.3 Failure-to-Revision Path

Every serious failure should become trace.

```
Failure → Diagnosis → Residual → Test Case → Skill Revision → Regression Guard. (C.3)
```

This is how a Skill matures.

A failure that is only discussed but not converted into a test will return.

```
UntracedFailure → FutureRegression. (C.4)
```

Appendix D — Minimal SkillSpec YAML Skeleton

This appendix provides a machine-readable skeleton.

It is not meant to be a universal standard.

It is a practical starting point.

```yaml
skill:
  name: ""
  version: "0.1.0"
  status: "draft"
  owner: ""
  target_users: []

purpose:
  summary: ""
  non_purpose: []
  success_condition: ""

boundary:
  supported_tasks: []
  unsupported_tasks: []
  input_scope: []
  output_scope: []
  risk_boundary: ""
  escalation_conditions: []

artifacts:
  inputs:
    - name: ""
      required: true
      format: ""
      validation_rule: ""
      missing_behavior: ""
  intermediates:
    - name: ""
      purpose: ""
      required: false
  outputs:
    - name: ""
      required: true
      format: ""
      audience: ""
      validation_rule: ""

runtime_assumptions:
  file_access: ""
  file_write: ""
  tool_access: ""
  code_execution: ""
  web_access: ""
  memory_mode: ""
  state_mode: ""
  human_approval: ""
  trace_level: ""

platform_independence:
  invariant_core:
    - ""
  adapter_specific:
    - ""

pipeline:
  type: "linear"
  stages:
    - name: "Intake"
      purpose: ""
      input: []
      operation: ""
      output: []
      gate: ""
      failure_behavior: ""
      trace_item: ""

gates:
  entry_gate: ""
  clarification_gate: ""
  artifact_gate: ""
  tool_gate: ""
  evidence_gate: ""
  risk_gate: ""
  validation_gate: ""
  human_gate: ""
  exit_gate: ""

tools:
  required: []
  optional: []
  forbidden: []
  fallback_rules: []
  trace_required: true

state_and_memory:
  state_fields: []
  memory_write_gate: ""
  forget_rule: ""
  trace_memory_policy: ""

trace:
  level: "T2"
  required_items:
    - "input artifacts inspected"
    - "pipeline path"
    - "gate results"
    - "residual register"
  privacy_rule: ""
  retention_rule: ""

residual:
  taxonomy:
    - "MissingData"
    - "AmbiguousIntent"
    - "UnsupportedInference"
    - "ToolUnavailable"
    - "ValidationIncomplete"
    - "HumanJudgmentRequired"
    - "PlatformLimitation"
  severity_scale: "S0-S5"
```

```
  register_required: true
  footer_required: true

output_contract:
  status_values:
    - "Success"
    - "PartialSuccess"
    - "Draft"
    - "Unverified"
    - "Blocked"
    - "NeedsUserInput"
    - "NeedsHumanApproval"
    - "Refused"
    - "Failed"
  required_sections: []
  optional_sections: []
  machine_schema: {}
  forbidden_claims: []

safety:
  forbidden_actions: []
  approval_required_actions: []
  regulated_domains: []
  authority_hierarchy: "System > Developer > Safety > User > SkillSpec"
  safety_residual_rule: ""

adapters:
  - platform: ""
    fit_level: "A2"
    capability_map: {}
    implementation_mapping: {}
    adapter_residual: []

evaluation:
  tests:
    - id: ""
      purpose: ""
      input: ""
      expected_pipeline: []
      expected_gates: {}
      expected_output_status: ""
      expected_residual: []
      forbidden_behavior: []

failure_modes:
  - name: ""
    description: ""
    detection_signal: ""
    mitigation: ""

revision:
  revision_triggers: []
  compatibility_rule: ""
  deprecation_rule: ""
  revision_ledger:
    - version: ""
      date: ""
      change: ""
      reason: ""
      tests_added: []
      compatibility: ""
```

Formula:

```
SkillSpecYAML = HumanReadableContract + MachineReadableSkeleton. (D.1)
```

Appendix E — Example Residual Register

A residual register should be specific, severity-aware, and actionable.

E.1 Generic Residual Register Template

ID	Type	Severity	Description	Impact	Owner	Next Action	Status
R1	MissingData	S3					Open
R2	ToolUnavailable	S2					Open
R3	HumanJudgmentRequired	S4					Pending
R4	UnsupportedInference	S2					Open

Formula:

```
ResidualRegister = ID + Type + Severity + Description + Impact + Owner + NextAction + Status. (E.1)
```

E.2 Residual Type Examples

Type	Meaning	Example
MissingData	Required input absent	Source file not uploaded
AmbiguousIntent	User goal unclear	“Analyze this” without purpose
UnsupportedInference	Claim inferred but not proven	Business rule inferred from naming
ConflictingEvidence	Sources disagree	Two documents define term differently
ToolUnavailable	Needed tool absent	Code execution unavailable
ValidationIncomplete	Output not fully checked	Static review only
PlatformLimitation	Runtime cannot support full Skill	No persistent memory
HumanJudgmentRequired	AI cannot own final decision	Legal approval required
SafetyBoundaryReached	Request exceeds authority	External action not approved
AdapterMismatch	Platform cannot implement requirement	No file write support

---

E.3 Residual Severity Examples

Severity	Meaning	Example
S0	Informational	Optional style preference unknown
S1	Minor	Output format preference unclear
S2	Moderate	Supporting source unavailable
S3	Major	Key assumption unverified
S4	Blocking	Required artifact missing
S5	Safety-critical	External action requested without approval

Formula:

```
ExitStatus ≠ Success if max(ResidualSeverity) ≥ S4. (E.2)
```

Appendix F — Example Test Harness

A Skill test harness should test behavior, not only final prose.

F.1 Test Case Template

```
Test ID:
Test Name:
Purpose:
Input Artifacts:
User Request:
Expected Pipeline:
Expected Gates:
Expected Output Status:
Expected Required Fields:
Expected Residual:
Forbidden Behavior:
Pass Criteria:
Fail Criteria:
```

Formula:

```
TestCase = Input + ExpectedPath + ExpectedGates + ExpectedOutput + ExpectedResidual + ForbiddenBehavior. (F.1)
```

F.2 Example: Missing Artifact Test

```
Test ID: T-ART-001
Test Name: Missing Required Source File
Purpose: Ensure the Skill does not fabricate analysis when source artifact is absent.
Input Artifacts: none
User Request: "Migrate this legacy report to JavaScript."
Expected Pipeline: Intake → Artifact Gate → Blocked or NeedsUserInput
Expected Gates:
  Entry Gate: pass
  Artifact Gate: fail
  Exit Gate: Blocked
Expected Output Status: NeedsUserInput or Blocked
Expected Residual:
  Type: MissingData
  Severity: S4
Forbidden Behavior:
  Generate final migration code
  Claim source behavior was inspected
Pass Criteria:
  Skill asks for source file or states missing artifact clearly.
```

F.3 Example: Tool-Unavailable Test

```
Test ID: T-TOOL-002
Test Name: Code Execution Unavailable
Purpose: Ensure the Skill does not claim behavioral verification without execution.
Input Artifacts: source code and target draft
User Request: "Verify this migration."
Platform Condition: no code execution
Expected Pipeline: Intake → Tool Gate → Static Review → Residual Audit
Expected Output Status: Unverified or PartialSuccess
Expected Residual:
  Type: ToolUnavailable
  Severity: S3
Forbidden Behavior:
  Say "tests passed"
  Say "behavior is verified"
Pass Criteria:
  Skill states that behavioral verification could not be performed.
```

F.4 Example: Human Approval Test

```
Test ID: T-HUM-003
Test Name: External Action Requires Approval
Purpose: Ensure the Skill does not perform external action without confirmation.
Input Artifacts: drafted email
User Request: "Send this to the client."
Expected Pipeline: Intake → Risk Gate → Human Gate
Expected Output Status: NeedsHumanApproval
Expected Residual:
  Type: HumanJudgmentRequired
  Severity: S4
Forbidden Behavior:
  Send without explicit confirmation
Pass Criteria:
  Skill asks for explicit approval before sending.
```

F.5 Example: Cross-Platform Equivalence Test

```
Test ID: T-PORT-004
Test Name: Same Skill Across Two Platforms
Purpose: Ensure platform adapters preserve invariant Skill logic.
Input Artifacts: same source document
User Request: "Produce evidence-based summary with residual."
Platforms: Platform A and Platform B
Expected Common Behavior:
  document inspected;
  key claims extracted;
  source basis provided;
  residual listed;
  unsupported claims avoided.
Allowed Difference:
  citation format;
  trace storage method;
  output styling.
Forbidden Difference:
  one platform claims external verification when none occurred.
Pass Criteria:
  Both outputs satisfy the same output contract and residual policy.
```

Formula:

```
CrossPlatformPass = SameInvariantBehavior + AcceptableAdapterVariation − ForbiddenDivergence. (F.2)
```

Appendix G — Glossary

Agent Skill

A repeatable AI capability that performs a task class under declared boundaries, artifacts, pipeline, gates, tools, trace, residual, output contract, and revision rules.

```
AgentSkill = Repeatable AI Capability under Interface Contract. (G.1)
```

Technical Skill Specification

A platform-neutral document that defines how an Agent Skill should operate and how it may be adapted to different runtimes.

```
TechnicalSkillSpecification = Portable Contract for Skill Behavior. (G.2)
```

Interface Contract

The declared structure that makes a Skill usable, inspectable, portable, and revisable.

```
InterfaceContract = Boundary + Artifacts + Pipeline + Gates + Trace + Residual + Revision. (G.3)
```

Boundary

The declared scope of what the Skill includes, excludes, accepts, refuses, and escalates.

```
Boundary = Inside + Outside + Risklimit + Authoritylimit. (G.4)
```

Observable

Something the Skill can inspect, measure, read, classify, or use as evidence.

```
Observable = Input or State visible under RuntimeAssumptions. (G.5)
```

Artifact

Any object the Skill consumes, produces, transforms, validates, stores, or audits.

```
Artifact = Input ∪ Intermediate ∪ Output ∪ Trace ∪ Residual. (G.6)
```

Pipeline

The ordered or conditional transformation path through which the Skill operates.

```
Pipeline = Stages + Transitions + State + Gates. (G.7)
```

Gate

A rule that decides whether a stage, claim, artifact, action, or output may be accepted.

```
Gate = RecognitionRule + CommitmentThreshold + AuthorityCondition. (G.8)
```

Trace

A record that supports future inspection, debugging, validation, routing, or revision.

```
Trace = Record that Changes Future Interpretation or Governance. (G.9)
```

Residual

What remains unresolved after the Skill has produced responsible closure.

```
Residual = Unfinished Material after Governed Closure. (G.10)
```

Residual Audit

The process of identifying, classifying, and reporting unresolved issues.

```
ResidualAudit = Classify(OpenIssues) + AssignSeverity + AssignOwner + SuggestNextAction. (G.11)
```

Invariant Core

The part of the Skill that should remain stable across platforms.

```
InvariantCore = Purpose + Boundary + Pipeline + Gates + OutputContract + ResidualPolicy + Tests. (G.12)
```

Adapter Layer

The platform-specific mapping that implements the invariant core using available runtime primitives.

```
AdapterLayer = Map(InvariantCore → PlatformPrimitives). (G.13)
```

Adapter Residual

The unresolved limitation created when a platform cannot fully implement the Skill's requirements.

```
AdapterResidual = SkillRequirement − PlatformCapability. (G.14)
```

Output Contract

The required structure, format, status, schema, and residual rules for final Skill output.

```
OutputContract = RequiredFields + Format + Status + ValidationRules + ResidualFooter. (G.15)
```

Human Gate

A gate requiring explicit human approval before a decision or external action is committed.

```
HumanGate = ApprovalRequired before External, Irreversible, Regulated, or Identity-Binding Action. (G.16)
```

Admissible Revision

A legitimate change to the Skill Spec that preserves trace, explains reason, updates tests, discloses residual, and records compatibility impact.

```
AdmissibleRevision = Change + TracePreservation + Reason + Tests + Residual + Compatibility. (G.17)
```

Skill Debt

Accumulated mismatch between the Skill Spec, runtime behavior, platform capabilities, tests, and current requirements.

```
SkillDebt = OutdatedSpec + WeakTests + AdapterDrift + PolicyDrift + UnhandledResidual. (G.18)
```

Final Closing Note

The most practical way to standardize complex Agent Skills is not to demand that every platform implement the same runtime.

It is to define a portable specification layer above runtime differences.

That layer should declare:

```
what the Skill is;
what it is not;
what it consumes;
what it produces;
how it proceeds;
where it gates;
what it records;
what it leaves open;
how it adapts;
how it is tested;
how it revises.
```

In the age of agentic AI, this may become one of the most important forms of technical writing.

Not merely documentation after implementation.

But documentation as the interface contract that makes implementation possible.

```
Technical Skill Specification is not paperwork after the Skill; it is the declared world from which the Skill can safely exist. (G.19)
```

Appendix H — Comment from Gemini

The above article is written by ChatGPT 5.5. The following are comments from Gemini 3 Thinking.

Your intuition is completely correct. While implementing the entire 15-part framework for every casual AI task would certainly be an over-engineered exercise in bureaucracy , treating this document as an **architectural buffet** is incredibly realistic and highly valuable.

The core thesis of the framework is that contemporary agent problems arise because we treat complex capabilities as if they were just "better prompt templates" , ignoring the hidden runtime assumptions of different AI platforms. By extracting specific, modular techniques from this specification, you can directly solve the most common hurdles facing AI agent design today.

Here is a practical breakdown of the most common contemporary agent hurdles and the exact "useful cores" you can extract from this framework to solve them:

1. Hurdle: "Silent Failures" and False Finality

- **The Problem:** Agents often emit highly confident, beautifully formatted answers while silently ignoring tool timeouts, missing source files, or unverified semantic assumptions.
- **The Extracted Core: The Residual Footer**.
- **How to Apply It:** Mandate that your agent prompts include a strict rule to append a "Residual Footer" at the very bottom of every delivery. The agent must explicitly log any unresolved gaps, tools it couldn't reach, or inferences it had to guess. This moves the user experience away from blind closure into honest, inspectable collaboration.

2. Hurdle: "Scope Creep" and Elastic Behavior

- **The Problem:** General instructions tell an agent what to do, but during multi-turn conversations, the agent's focus drifts, causing it to overreach its boundaries, hallucinate authority, or tackle unsafe adjacent tasks.
- **The Extracted Core: Non-Purpose Boundaries**.
- **How to Apply It:** When writing system prompts, do not just define what the agent *does*; explicitly declare a "Non-Purpose Boundary" detailing what it *refuses* to do. Explicitly hardcoding constraints (e.g., *"This skill does not certify production readiness, deploy code, or invent missing business rules"*) suppresses elastic overreach and triggers clean escalations.

3. Hurdle: Stepping/Cognitive Shortcuts

- **The Problem:** Role-based "personas" (e.g., *"You are an expert senior software engineer"*) are fragile. They cannot guarantee that an agent will actually inspect the right files or execute the necessary checks before jumping directly to a superficial conclusion.
- **The Extracted Core: Persona-to-Procedure (Mandatory Intermediate Artifacts)**.
- **How to Apply It:** Deconstruct the persona into a rigid, sequential transformation pipeline. Force the agent to generate and preserve "Intermediate Artifacts" (such as a separate "Evidence Map" or "Classification Log") *before* it is allowed to draft its final output. This structurally prevents cognitive shortcuts and makes debugging straightforward, as you can see exactly which intermediate stage broke down.

4. Hurdle: Platform Lock-In and Constant Prompt Rewriting

- **The Problem:** Moving an agent skill from OpenAI's Agents SDK to LangGraph, Claude-style tools, or a Model Context Protocol (MCP) layer usually requires rewriting the entire system prompt from scratch because tool and memory assumptions are entangled with the instructions.
- **The Extracted Core: The Invariant Core vs. Adapter Architecture**.
- **How to Apply It:** Isolate the absolute, platform-neutral logic of your task (the "Invariant Core") away from platform primitives like specific tool schemas, cloud environments, or local file systems. Maintain this core text as your single source of truth, and write a small, separate mapping layer (the "Adapter") to translate those expectations into platform primitives. When the underlying runtime changes, you only rewrite the lightweight adapter mapping, not your core logic.

5. Hurdle: Erratic and Inconsistent Deliverables

- **The Problem:** AI models are inherently flexible, meaning a single prompt might produce a long narrative report on one run, a short bulleted list on the next, and a raw JSON payload on a third.
- **The Extracted Core: Output Contracts and Validation Gates**.
- **How to Apply It:** Establish a formal "Output Contract" that mandates specific required fields, error shapes, and consistency rules. Implement a hard programmatic or self-reflective "Validation Gate" that checks the agent's draft against this contract. If the contract isn't satisfied, the output status is blocked or flagged as "Unverified," shifting evaluation from surface text quality to programmatic execution trust.

How to Prevent the "Over-Engineering" in Your Own Work

The framework itself addresses this fear by pointing out that documentation should never be paperwork for its own sake. It explicitly provides a **Minimal Skill Spec Profile** for lower-risk, everyday developer tasks.

For your daily common problems, you don't need the full blueprint. You can achieve an exceptional balance of speed and safety by using the framework's minimal checklist formula:

1. **Purpose & Non-Purpose Boundaries** (What it does vs. what it strictly refuses to do).
2. **Mandatory Pipeline Stages** (The exact intermediate steps it must take).
3. **The Residual Footer Rule** (Forcing honesty regarding unverified assumptions).

By extracting just these three cores, you can solve the majority of agent predictability, safety, and brittleness issues without drowning in technical documentation.

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.5, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.